

2주차 개념교안

version.24.08.14

해당 교안을 무단으로 복제, 업로드, 배포, 도용할 경우

저작권법 제 97조의 저작권침해죄에 해당하며

5년이하의 징역 또는 5천만원 이하의 벌금에 처해질 수 있습니다.

그래프이론.....	5
정점과 간선.....	5
indegree outdegree.....	7
가중치.....	7
트리.....	7
트리의 특징.....	8
루트노드.....	9
내부노드.....	9
리프노드.....	9
트리의 높이와 레벨.....	10
이진트리(BT, Binary Tree).....	11
균형이진트리의 높이 차이 계산.....	11
이진탐색트리(BST, Binary Search Tree).....	12
Q. 이진탐색트리의 시간복잡도는 얼마일까요?.....	13
그래프 구현과 탐색 1. 인접행렬.....	15
문제로 알아가는 인접행렬.....	18
Q1. 3번노드에서 5번노드로 가는 단방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?.....	18
Q2. 3번노드에서 5번노드로 가는 양방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?.....	18
Q3. 정점의 갯수가 20개인 그래프가 있습니다. 이를 인접행렬로 표현한다고 하고 메모리를 최소로 쓴다고 했을 때 배열을 어떻게 만들어야 할까요?.....	19
Q4. 노드 1, 2, 3이 있으며, 1번 노드에서 2번 노드로 가는 단방향 경로와 2번 노드에서 3번 노드로 가는 단방향 경로가 있습니다. 이를 인접행렬로 표현한다면 어떻게 될까요?.....	19
Q5. 1 ~ 4번노드로 이루어진 4개의 노드가 있습니다. 1번 노드에서 2번 노드로 가는 경로, 2번 노드에서 3번 노드로 가는 경로, 3번 노드에서 4번 노드로 가는 경로, 그리고 4번 노드에서 1번 노드로 가는 경로가 있을 때, 인접행렬은 어떻게 나타날까요?.....	20
Q6. 1 ~ 5번노드로 이루어진 5개의 노드가 있습니다. 모든 노드가 서로 연결되어 있는 완전 그래프의 인접행렬은 어떻게 될까요?.....	20
Q7. 노드 1, 2, 3이 있으며, 1번 노드에서 3번 노드로 가는 단방향 경로가 있고, 3번 노드에서	

2번 노드로 가는 단방향 경로가 있습니다. 또한, 2번 노드에서 다시 1번 노드로 가는 경로가 있습니다. 이 경우 인접행렬은 어떻게 될까요?.....	21
인접행렬을 기반으로 탐색하기.....	21
그래프 구현과 탐색 2. 인접리스트.....	22
Q. 왜 list가 아닌 vector로 구현할까?.....	24
list의 시간복잡도.....	24
vector의 시간복잡도.....	24
Q. 인접리스트를 기반으로 탐색하기.....	25
인접행렬과 인접리스트의 차이.....	26
공간복잡도.....	26
시간복잡도 : 간선 한개 찾기.....	26
시간복잡도 : 모든 간선찾기.....	27
그래프가 희소할 때는 인접리스트, 조밀할 때는 인접행렬을 써라.....	27
정리.....	28
그래프 구현과 탐색 3. 맵과 방향벡터.....	28
4방향탐색과 방향벡터.....	29
Q1. {0, 0}좌표에서 dy, dx를 만들어 4방향(위, 오른쪽, 아래, 왼쪽)을 탐색하며 좌표를 출력하시오.....	30
Q2. {0, 0}좌표에서 dy, dx를 만들어 8방향(위, 오른쪽, 아래, 왼쪽 및 대각선방향포함)을 탐색하며 좌표를 출력하시오.....	30
Q3. 3 * 3 맵을 입력받아야 함. 이 맵은 1과 0으로 이루어져있고 {0, 0}은 무조건 1임을 보장한다. {0, 0}부터 4방향을 기준으로 한칸씩 탐색해나가며 방문한 정점은 다시 방문하지 않으며 방문하는 좌표를 출력하는 코드. 0은 갈 수 없는 지역. 1은 갈 수 있는 지역을 구현하시오.....	31
연결된 컴포넌트(connected component).....	32
깊이우선탐색.....	34
DFS의 수도코드.....	34
구현방법1 : 돌다리를 두들겨 보다.....	37
구현방법2 : 못 먹어도 GO.....	38
Q. 종화는 방구쟁이야!.....	39
디버깅 팁.....	42

너비우선탐색(BFS, Breadth First Search).....	43
BFS의 수도코드1.....	45
BFS의 수도코드2.....	45
Q. 만약 앞의 그래프의 가중치가 한칸이 아니고 2칸이라면 어떻게 될까요?.....	48
Q. 시작지점이 다수라면?.....	48
Q. BFS는 왜 가중치가 같은 그래프내에서만 최단거리 알고리즘으로 쓰이나요? 가중치가 다른 그래프에서는 안되나요?.....	49
Q. 당근마켓 엔지니어 승원이.....	50
DFS와 BFS비교.....	53
트리순회.....	54
후위순회.....	54
전위순회.....	54
중위순회.....	55
레벨순회.....	55
Q. 아래의 그래프가 주어졌을 때 preorder, inorder, postorder를 구현하라.....	55
정리.....	58

이번 2주차는 그래프이론과 DFS(깊이우선탐색), BFS(너비우선탐색) 그리고 트리순회인 preorder, inorder, postorder를 다루겠습니다.

그래프이론

그래프이론은 오일로경로, SCC, 단절점 등의 어려운 개념들도 많고 넓은 범위를 다루는 이론이지만

오늘은 기초중의 기초를 배워보겠습니다.

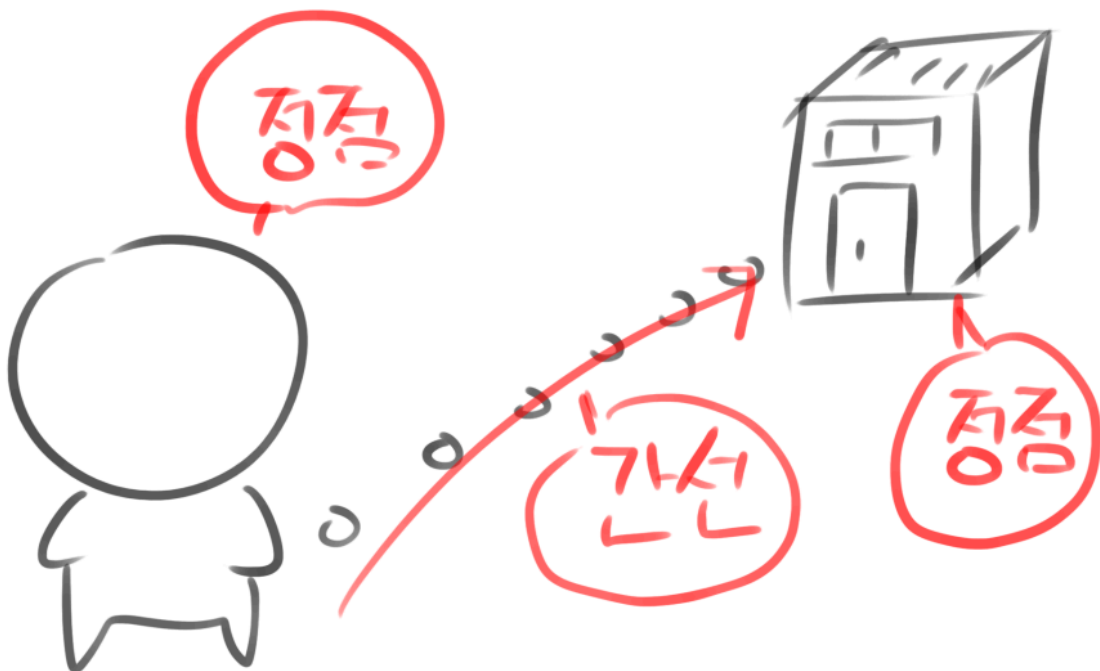
정점과 간선

정점(vertex)은 노드라고도 불리며 그래프를 형성하는 기본 단위입니다.

정점은 분할할 수 없는 객체이자 "점"으로 표현되는 위치, 사람, 물건 등이 될 수 있습니다.

간선(Edge)은 정점을 잇는 선을 의미합니다. 관계, 경로 등이 될 수 있습니다.

예를 들어 "어떠한 위치나 어떠한 사람"으로부터 "무언가를 통해 간다"라고 했을 때 "어떠한 위치나 어떠한 사람" 정점(Vertex)이 되고 "무언가를 통해 간다"는 간선(Edge)이 됩니다.



필자가 위 그림처럼 어떤 아파트로 간다고 해봅시다. 저와 아파트는 하나의 정점이고 거기서 가는 길은 간선이 됩니다.



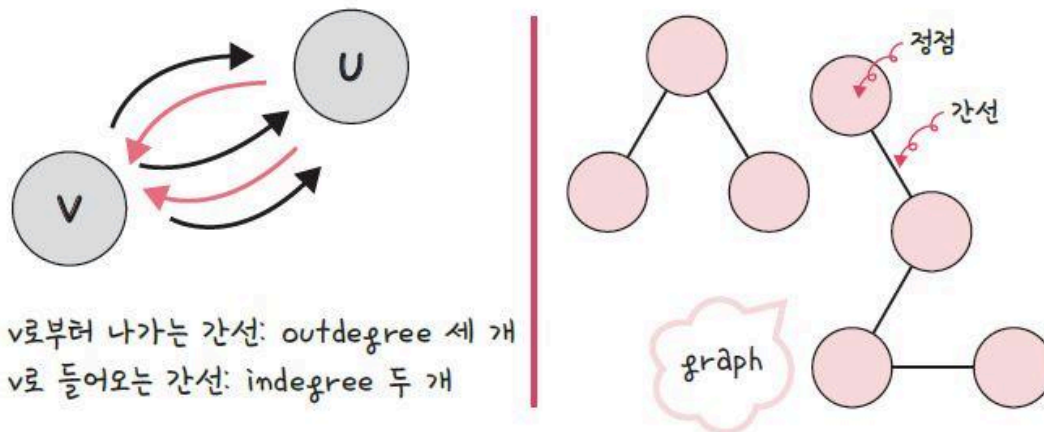
필자가 어떤 사람을 좋아한다고 해봅시다. 필자와 어떤 사람은 정점이 이러한 마음은 간선이 됩니다. 자 이 때 제가 만약에 어떤 사람을 좋아하는데 어떤 사람은 저를 좋아하지 않는 "짝사랑"이라고 해봅시다. 이것은 어떠한 간선일까요? 바로 단방향 간선입니다. 저의 마음만이 그 사람으로 가는 길밖에 없는 슬픈 관계이지요.



하지만 그 어떤 사람도 저를 좋아한다면 어떻게 될까요? 바로 양방향 간선이 됩니다. 저의 마음도 그 어떤 사람의 마음도 갈 수 있는 "사랑"이라 불리는 상태인 것이죠.

indegree outdegree

▼ 그림 5-12 그래프



정점으로 나가는 간선을 해당 정점의 outdegree라고 하며 들어오는 간선을 해당정점의 indegree라고 합니다. 해당 정점은 outdegree는 3개 indegree는 2개인 상태입니다. 또한 정점은 약자로 V또는 U라고 하며 보통 어떤 정점으로부터 시작해서 어떤 정점까지를 간다를 "U에서부터 V로 간다" 라고 표현을 합니다.

가중치

정점과 정점사이에 드는 비용을 뜻합니다. 1번노드와 2번노드까지 가는 비용이 한칸이라면 1번노드에서 2번노드까지의 가중치는 한칸입니다.

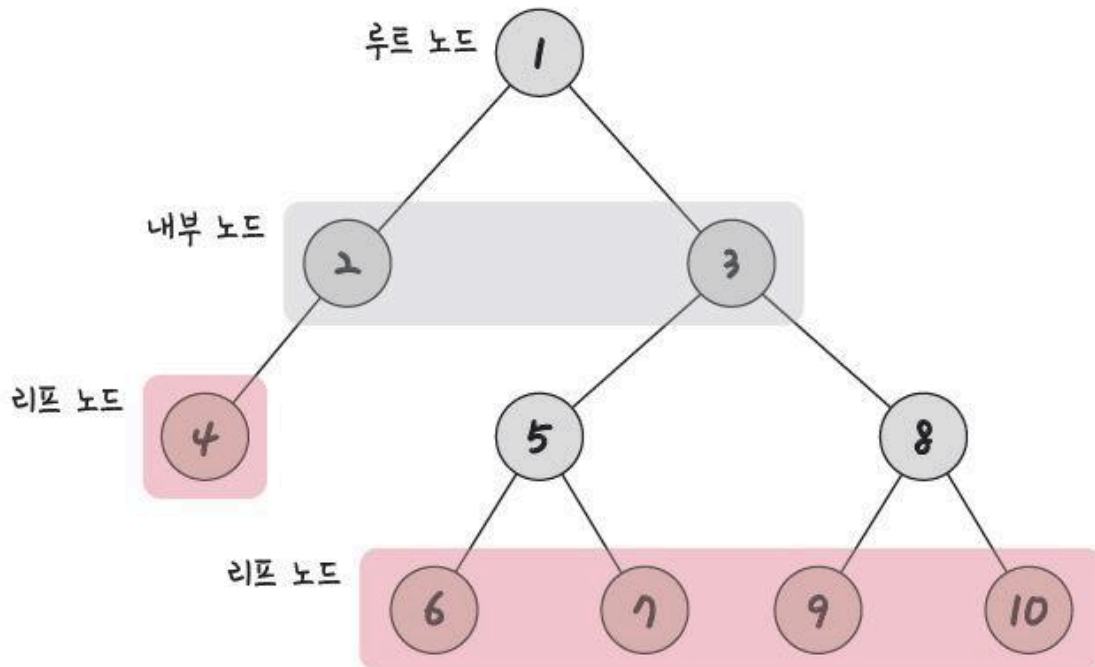
예를 들어 제가 성남이라는 정점에서 네이버라는 정점까지 걸리는 택시비가 13000원이라면 성남에서 네이버까지 가는 가중치는 13000원이 됩니다.

지금까지 설명한 정점과 간선들로 이루어진 집합을 "그래프(Graph)"라고 합니다.

트리

트리는 자식노드와 부모노드로 이루어진 계층적인 구조를 가지며 무방향 그래프의 일종이자 사이클이 없는 자료구조를 의미합니다.

▼그림 5-13 트리의 특징



트리의 특징

트리는 그래프의 일종이며 다음 특징을 가집니다.

1. 부모, 자식 계층 구조를 가집니다. 지금 보면 5번 노드는 6번 노드와 7번 노드의 부모 노드이고, 6번노드와 7번 노드는 5번 노드의 자식 노드입니다. 같은 경로 상에서 어떤 노드보다 위에 있으면 부모,아래에 있으면 자식 노드가 됩니다.
2. $V - 1 = E$ 라는 특징이 있습니다. 간선 수는 노드 수 - 1입니다.
3. 임의의 두 노드 사이의 경로는 '유일무이'하게 '존재'합니다. 즉, 트리 내의 어떤 노드와 어떤 노드까지의 경로는 반드시 있으며 하나밖에 없습니다.

루트노드

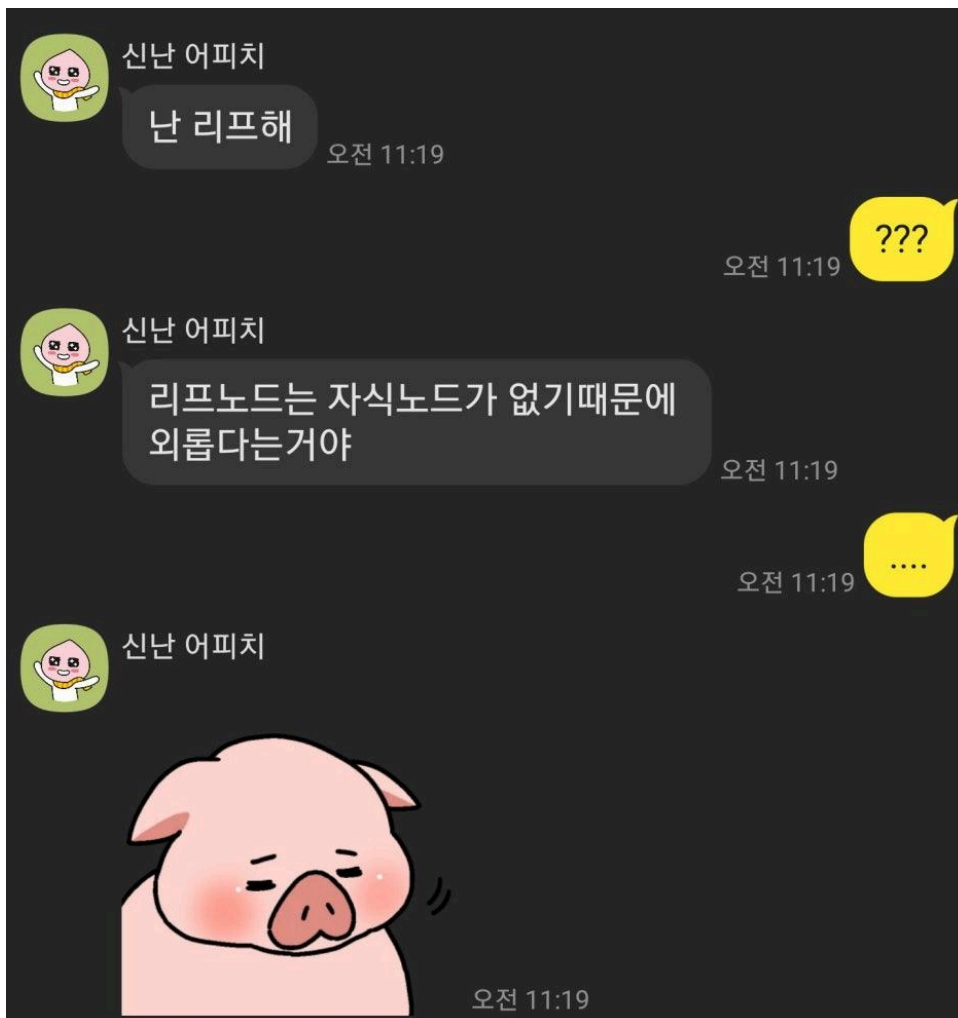
가장 위에 있는 노드를 뜻합니다. 보통 트리문제가 나오고 트리를 탐색할 때 루트노드를 중심으로 탐색을 하면 문제가 쉽게 풀리는 경우가 많습니다.

내부노드

루트노드와 리프노드 사이에 있는 노드를 뜻합니다.

리프노드

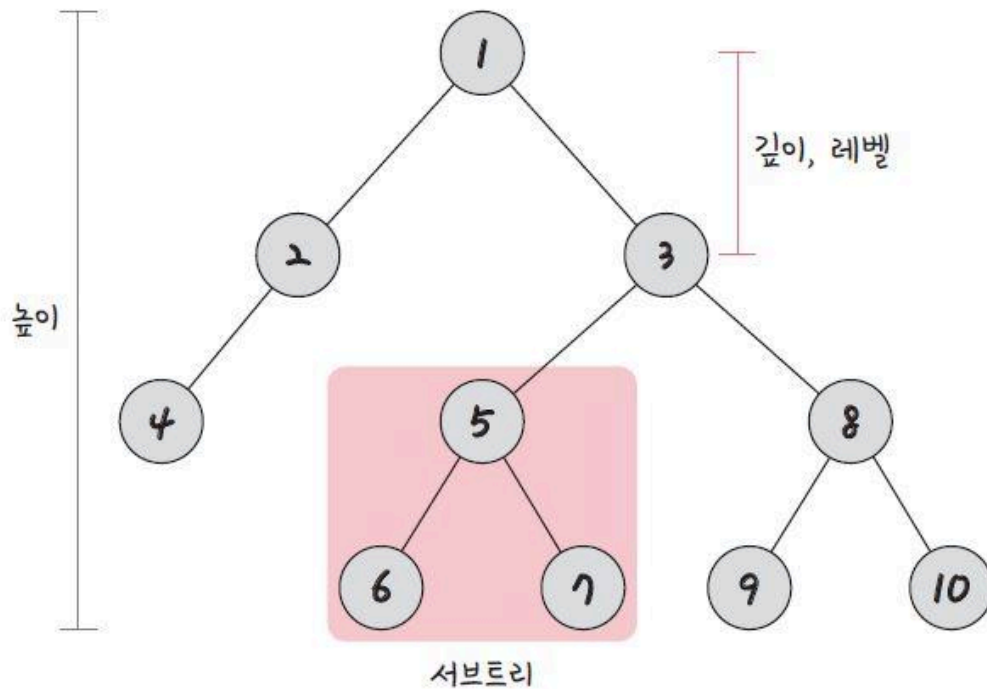
실제 알고리즘 고인물과 카톡한 내용입니다. 아래그림처럼 리프노드는 자식노드가 없는 노드를 뜻합니다.



트리의 높이와 레벨

다음 그림은 트리의 높이와 레벨을 설명한 그림입니다.

▼그림 5-15 트리의 높이와 레벨



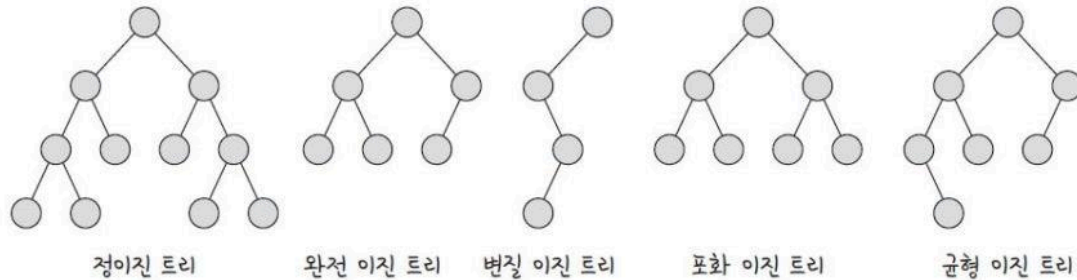
- 깊이: 트리에서의 깊이는 각각의 노드마다 다르며 루트 노드에서 특정 노드까지 최단거리로 갔을 때의 거리를 말합니다. 예를 들어 4번 노드의 깊이는 2입니다.
- 높이: 트리의 높이는 루트 노드부터 리프 노드까지의 거리 중 가장 긴 거리를 의미하며 앞 그림의 트리 높이는 3입니다.
- 레벨: 트리의 레벨은 주어지는 문제마다 조금씩 다르지만 보통 깊이와 같은 의미를 지닙니다. 1번 노드를 0레벨이라고 하고 2번 노드, 3번 노드까지의 레벨을 1레벨이라고 할 수도 있고, 1번 노드를 1레벨이라고도 할 수 있습니다. 예를 들어 1번 노드를 1레벨이라고 한다면 2번 노드와 3번 노드는 2레벨이 됩니다.
- 서브트리: 트리 내의 하위 집합을 서브트리라고 합니다. 트리 내에 있는 부분집합이라고도 보면 됩니다. 지금 보면 5번, 6번, 7번 노드가 이 트리의 하위 집합으로 "저 노드들은 서브트리다"라고 합니다.

[참고] 트리로 이루어진 집합을 숲(forest)이라고 합니다.

이진트리(BT, Binary Tree)

각각의 노드의 자식노드 수가 2개 이하로 구성되어있는 트리를 의미하며 이를 다음과 같이 분류합니다.

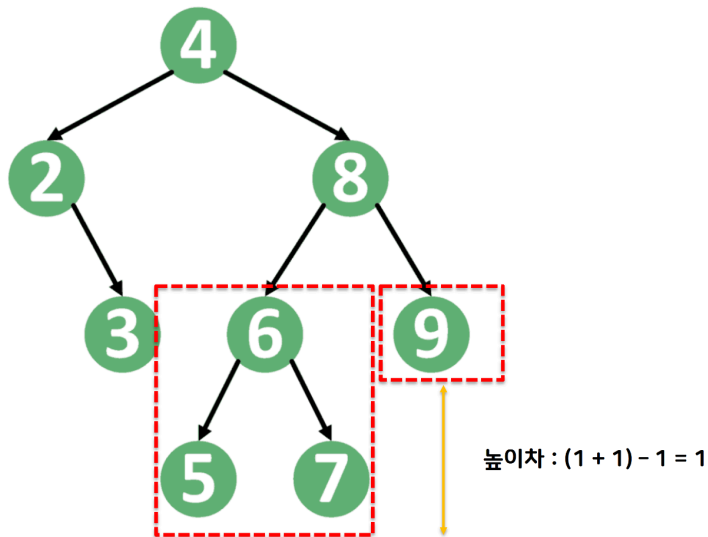
▼ 그림 5-16 트리의 종류



- 정이진 트리(full binary tree): 자식 노드가 0 또는 2개인 이진 트리를 의미합니다.
- 완전 이진 트리(complete binary tree): 왼쪽에서부터 채워져 있는 이진 트리를 의미합니다. 마지막 레벨을 제외하고는 모든 레벨이 완전히 채워져 있으며 마지막 레벨의 경우 왼쪽부터 채워져 있습니다.
- 변질 이진 트리(degenerate binary tree): 리프노드 제외, 자식 노드가 하나밖에 없는 이진 트리를 의미합니다.
- 포화 이진 트리(perfect binary tree): 모든 노드가 짝 차 있는 이진 트리를 의미합니다.
- 균형 이진 트리(balanced binary tree): 균형 이진 트리는 트리의 모든 노드에 대해, 그 노드의 왼쪽 하위 트리와 오른쪽 하위 트리의 높이 차이가 1 이하인 이진 트리입니다. 이 말은, 트리의 어느 한쪽이 지나치게 치우치지 않도록 구조가 유지된다는 것을 의미합니다. 이러한 균형 덕분에, 탐색, 삽입, 삭제 등의 연산에서 시간 복잡도가 $O(\log n)$ 으로 보장될 수 있습니다.

균형이진트리의 높이 차이 계산

높이차이는 각 하위 트리의 전체 높이를 기준으로 계산합니다. 예를 들어 한 노드의 왼쪽 하위 트리의 높이가 1이고, 오른쪽 하위 트리의 높이가 0이라면, 이 노드의 하위 트리 높이 차이는 1이 됩니다.

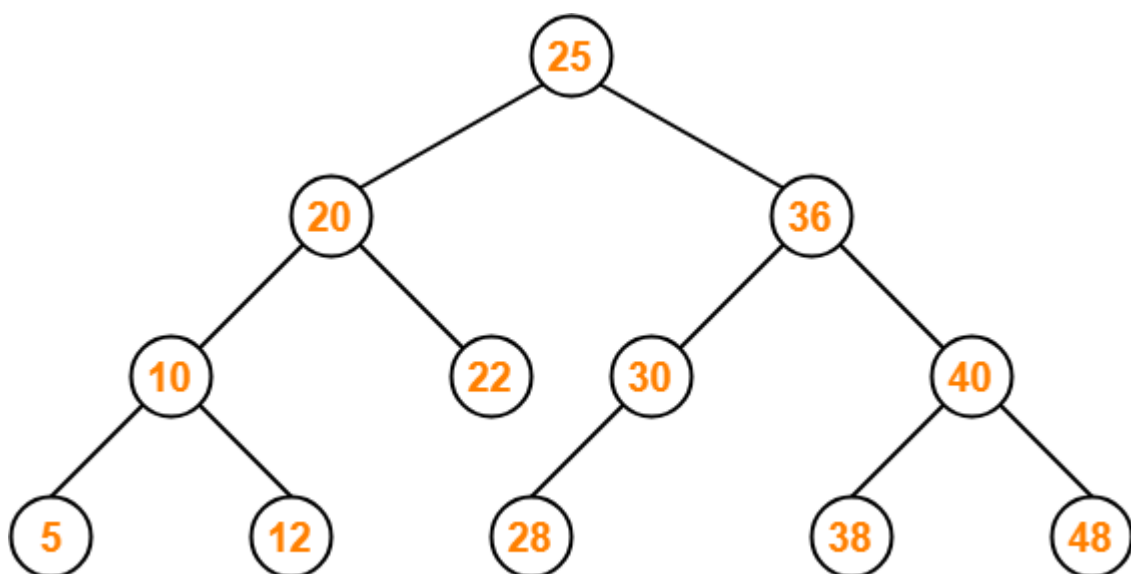


이 차이가 1 이하로 유지되도록 트리를 구성하면, 그 트리는 균형 이진 트리가 됩니다.

[참고] 레드-블랙 트리(Red-Black Tree)는 균형 이진 트리의 한 예로, 자가 균형을 유지하기 위한 특정한 규칙들을 갖춘 이진 탐색 트리입니다. 이 규칙들 덕분에 레드-블랙 트리는 최악의 경우에도 효율적인 연산을 보장하며, C++의 `std::map`과 `std::set` 같은 데이터 구조의 내부 구현에 사용됩니다.

이진탐색트리(BST, Binary Search Tree)

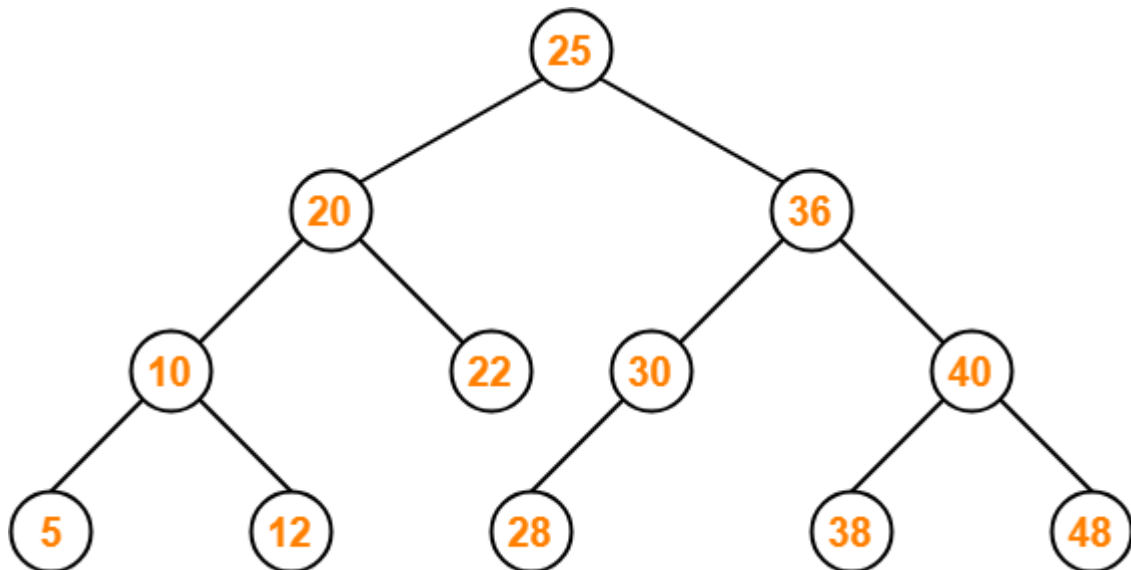
다음 그림은 이진탐색트리(BST) 입니다. 이진트리의 일종으로 노드의 오른쪽 하위 트리에는 "노드의 값보다 큰 값"이 있는 노드만 포함되고 왼쪽 하위트리에는 "노드의 값보다 작은값"이 들어있는 트리를 말합니다.



이 때 왼쪽 및 오른쪽 하위 트리도 해당 특성을 가집니다. 이렇게 두면 "검색"을 하기에 용이합니다. 왼쪽에는 작은 값, 오른쪽에는 큰 값이 이미 정해져있기 때문에 예를 들어 10을 찾으려고 한다면 25의 왼쪽 노드들만 찾으면 된다는 것은 자명하기 때문입니다. 이렇게 한다면 전체탐색을 하지 않아도 되겠죠?

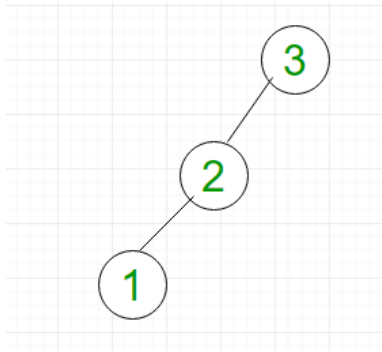
Q. 이진탐색트리의 시간복잡도는 얼마일까요?

만약 이렇게 균형잡히게 분포가 되었다면 탐색, 삽입, 삭제, 수정 모두 $O(\log N)$ 입니다.

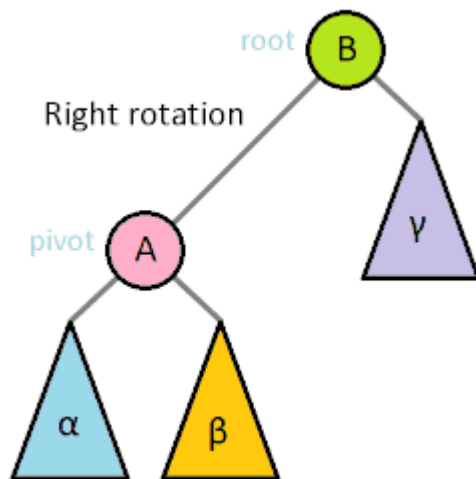


그러나. 이는 삽입 순서에 따라 달라집니다. 예를 들어 1, 2, 3 이렇게 삽입이 되어 이진탐색트리가 완성이 되었다면. 이렇게 "선형적"인 자료구조가 완성되어버립니다.

참고로 선형적이란 직선, 비선형적은 직선이 아닌 모양을 의미합니다.

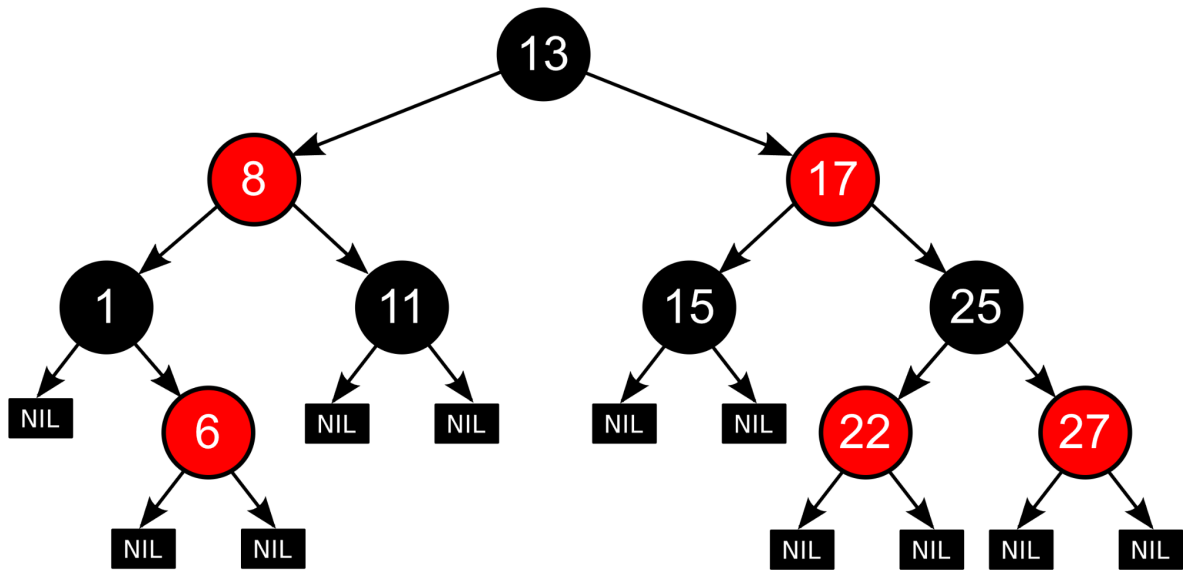


즉, 이진탐색트리는 삽입순서에 따라 영향을 받습니다. 그러나 삽입순서가 어떻게 되든 트리의 노드들을 회전시키는 등의 방법을 통해서 "균형잡히게 만든" 이진탐색트리에서 발전된 트리로는 AVL트리, 레드블랙트리가 있습니다.



[AVL 트리]

map이라는 자료구조는 삽입, 탐색, 삭제, 수정의 시간복잡도가 모두 $O(\log N)$ 임을 보장받는데 그 이유가 균형잡힌 트리인 레드블랙트리를 기반으로 구현되어있기 때문입니다.



[레드블랙트리]

그래프 구현과 탐색 1. 인접행렬

자, 지금까지 그래프에 대해서 얘기를 해봤는데 어떻게 이를 컴퓨터에게 알려줄까요?

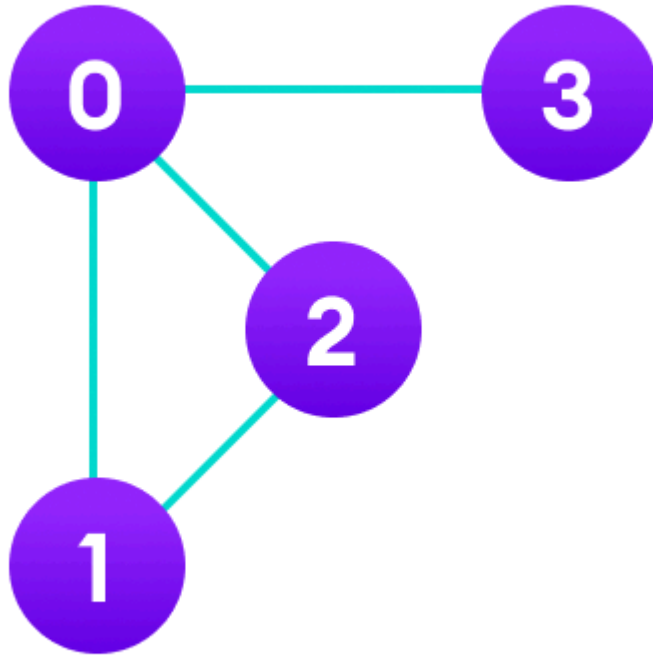
컴퓨터에게 내가 이러한 그래프를 그렸다고 알려줄 표현방법으로는 인접행렬과 인접리스트가 있습니다.

[참고] 인접해있다 = 연결되어있다. 라고 보면 됩니다.

먼저 인접행렬에 대해 설명해보겠습니다.

인접행렬이란 그래프에서 정점과 간선의 관계를 나타내는 bool 타입의 정사각형 행렬을 의미합니다. 정사각형 행렬의 각 요소가 0 또는 1이라는 값으로 가짐을 의미하는데요. 0은 두 정점 사이의 경로가 없음을 의미하며 1은 두 정점 사이의 경로가 있음을 의미합니다.

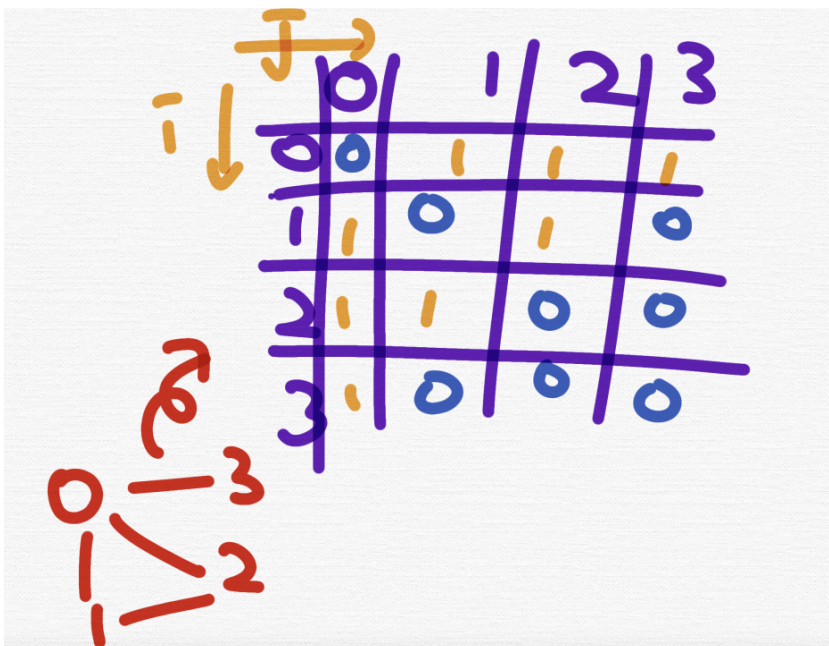
예를 들어 아래와 같은 그래프가 있습니다. 이 그래프는 무방향 그래프로 설명되어있는데 무방향 그래프 := 양방향 그래프로 봐도 됩니다.



이를 다음과 같은 행렬로 표현하는 것을 말합니다.

1 - 1, 2 - 2를 보면 0으로 되어있는 것을 볼 수 있는데 자기자신을 나타내는 것이며 해당 정점의 사이클이 없을 때는 0, 사이클이 있을 때는 1로 표기를 합니다.

0 - 1 이 연결되어있기 때문에 $a[0][1] = 1$ 이 됩니다. 그러나 1 - 3은 연결되어있지 않기 때문에 $a[1][3] = 0$ 이 됩니다.



이걸 코드로 표현하면 다음과 같습니다.

```
bool a[4][4] = {
    {0, 1, 1, 1},
    {1, 0, 1, 0},
    {1, 1, 0, 0},
    {1, 0, 0, 0},
};
```

그렇다면 이를 기반으로 어떻게 코드를 구축할까요?

보통은 다음코드처럼 2중 for문을 통해 i에서 j로 가는 경로가 있다면 어떠한 로직 또는 해당 정점으로 부터 탐색하는 로직을 구축합니다. 참고로 BFS와 DFS는 하나의 예에 불과합니다. 이것들 말고도 이 정점으로부터 탐색하는 다양한 알고리즘을 쓸 수 있습니다.

```
bool a[V][V];
for(int i = 0; i < V; i++){
    for(int j = 0; j < V; j++){
        if(a[i][j]){
            //출력하는 로직
            cout << i << "부터 " << j << "까지 경로가 있습니다.\n";
            // 해당 정점으로 부터 탐색하는 로직
            BFS(i);
            DFS(i);
        }
    }
}
```

자 그러면 문제를 풀면서 인접행렬을 더 알아가보도록 하겠습니다.

문제로 알아가는 인접행렬

Q1. 3번노드에서 5번노드로 가는 단방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?

답 :

```
a[3][5] = 1;
```

Q2. 3번노드에서 5번노드로 가는 양방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?

답 :

```
a[3][5] = 1;  
a[5][3] = 1;
```

Q3. 정점의 갯수가 20개인 그래프가 있습니다. 이를 인접행렬로 표현한다고 하고 메모리를 최소로 쓴다고 했을 때 배열을 어떻게 만들어야 할까요?

답 :

```
bool a[20][20];
```

Q4. 노드 1, 2, 3이 있으며, 1번 노드에서 2번 노드로 가는 단방향 경로와 2번 노드에서 3번 노드로 가는 단방향 경로가 있습니다. 이를 인접행렬로 표현한다면 어떻게 될까요?

답:

```
  1  2  3
1 [0, 1, 0]
2 [0, 0, 1]
3 [0, 0, 0]
```

Q5. 1 ~ 4번노드로 이루어진 4개의 노드가 있습니다. 1번 노드에서 2번 노드로 가는 경로, 2번 노드에서 3번 노드로 가는 경로, 3번 노드에서 4번 노드로 가는 경로, 그리고 4번 노드에서 1번 노드로 가는 경로가 있을 때, 인접행렬은 어떻게 나타날까요?

답:

	1	2	3	4
1	[0, 1, 0, 0]			
2	[0, 0, 1, 0]			
3	[0, 0, 0, 1]			
4	[1, 0, 0, 0]			

Q6. 1 ~ 5번노드로 이루어진 5개의 노드가 있습니다. 모든 노드가 서로 연결되어 있는 완전 그래프의 인접행렬은 어떻게 될까요?

답:

	1	2	3	4	5
1	[0, 1, 1, 1, 1]				
2	[1, 0, 1, 1, 1]				
3	[1, 1, 0, 1, 1]				
4	[1, 1, 1, 0, 1]				
5	[1, 1, 1, 1, 0]				

[참고] 완전 그래프에서는 자기 자신을 가리키는 경로가 없으므로 대각선은 0이고, 나머지 모든 요소는 1입니다.

Q7. 노드 1, 2, 3이 있으며, 1번 노드에서 3번 노드로 가는 단방향 경로가 있고, 3번 노드에서 2번 노드로 가는 단방향 경로가 있습니다. 또한, 2번 노드에서 다시 1번 노드로 가는 경로가 있습니다. 이 경우 인접행렬은 어떻게 될까요?

답:

```
   1  2  3
1 [0, 0, 1]
2 [1, 0, 0]
3 [0, 1, 0]
```

인접행렬을 기반으로 탐색하기

1번.

정점은 0번 부터 9번까지 10개의 노드가 있다. 1 - 2 / 1 - 3 / 3 - 4 라는 경로가 있다. (1번과 2번, 1번과 3번, 3번과 4번은 연결되어있다.) 이를 인접행렬로 표현하라.

2번.

0번부터 방문안한 노드를 찾고 해당 노드부터 방문, 연결된 노드를 이어서 방문해서 출력하는 재귀함수를 만들어라. 여기서 정점을 방문하고 다시 방문하지 않게 만들어야 한다.

정답코드

```
#include<bits/stdc++.h>
using namespace std;
const int V = 10;
bool a[V][V], visited[V];
void go(int from){
    visited[from] = 1;
    cout << from << '\n';
    for(int i = 0; i < V; i++){
        if(visited[i]) continue;
        if(a[from][i]){
            go(i);
        }
    }
}
```

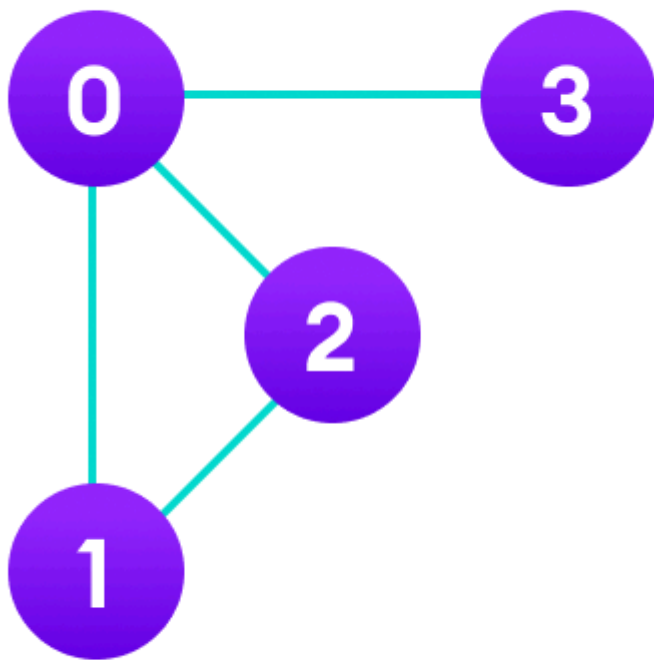
```

    return;
}
int main(){
    a[1][2] = 1; a[1][3] = 1; a[3][4] = 1;
    a[2][1] = 1; a[3][1] = 1; a[4][3] = 1;
    for(int i = 0; i < V; i++){
        for(int j = 0; j < V; j++){
            if(a[i][j] && visited[i] == 0){
                go(i);
            }
        }
    }
}

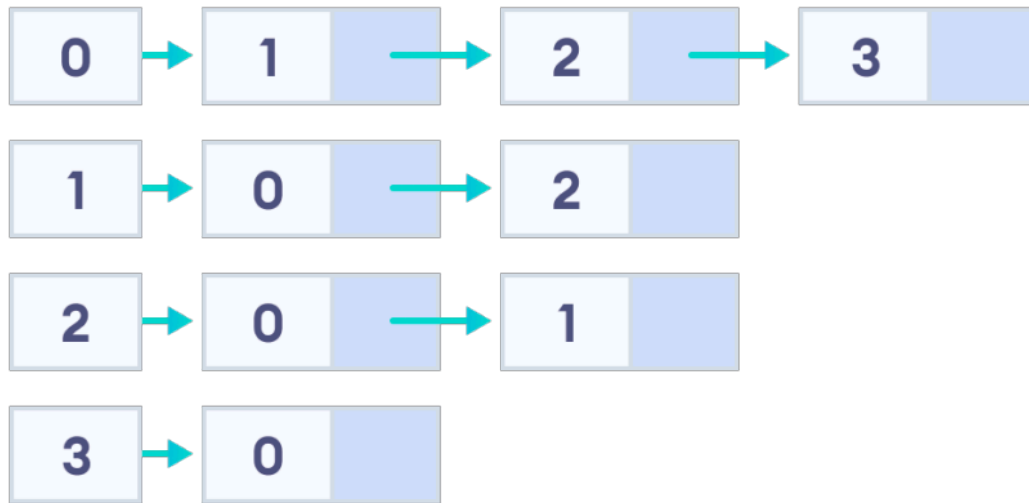
```

그래프 구현과 탐색 2. 인접리스트

인접 리스트(adjacency list)는 그래프에서 정점과 간선의 관계를 나타내는 연결리스트를 의미합니다. 예를 들어 아래와 같은 그래프가 있습니다.



이 그래프를 아래와 같이 나타낼 수 있습니다.



이를 코드로 나타내면 다음과 같습니다.

```

#include<bits/stdc++.h>
using namespace std;
const int V = 4;
vector<int> adj[V];
int main(){
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[0].push_back(3);

    adj[1].push_back(0);
    adj[1].push_back(2);

    adj[2].push_back(0);
    adj[2].push_back(1);

    adj[3].push_back(0);

    for(int i = 0; i < 4; i++){
        cout << i << " :: ";
        for(int there : adj[i]){
            cout << there << " ";
        }
        cout << '\n';
    }
    // 이렇게도 할 수 있다.
  
```

```

    for(int i = 0; i < 4; i++){
        cout << i << " :: ";
        for(int j = 0; j < adj[i].size(); j++){
            cout << adj[i][j] << " ";
        }
        cout << '\n';
    }

}
/*
0 :: 1 2 3
1 :: 0 2
2 :: 0 1
3 :: 0
*/

```

Q. 왜 list가 아닌 vector로 구현할까?

앞의 코드는 vector로 구현했는데 인접리스트인 list로 구현해도 되고 vector로 구현해도 됩니다.

```

// vector
vector<int> adj[1004];
// list
list<int> adj[1004];

```

그러나 보통 vector를 주로 많이 씁니다. 이는 list 와 vector의 연산 시간복잡도 때문입니다.

list의 시간복잡도

- n번째 인덱스에 삽입, 삭제 : $O(1)$
- 마지막 요소에 삽입, 삭제 : $O(1)$
- 특정요소 탐색 : $O(n)$
- n번째 요소 참조 : $O(n)$

vector의 시간복잡도

- n번째 인덱스에 삽입, 삭제 : $O(n)$
- 마지막 요소에 삽입, 삭제 : $O(1)$

- 특정요소 탐색 : $O(n)$
- n번째 요소 참조 : $O(1)$

인접리스트를 기반으로 로직을 구현할 때 많이 사용되는 연산은 마지막 요소에 삽입과 해당 배열의 n번째 요소를 탐색하는 연산입니다. 그렇기 때문에 vector로 구현하는게 좋습니다.

Q. 인접리스트를 기반으로 탐색하기

1번과 2번을 만족시키는 코드를 만들어내야 합니다.

1번.

정점은 0번 부터 9번까지 10개의 노드가 있다. 1 - 2 / 1 - 3 / 3 - 4 라는 경로가 있다. (1번과 2번, 1번과 3번, 3번과 4번은 연결되어있다.)이를 인접리스트로 표현한자?

2번.

0번부터 방문안한 노드를 찾고 해당 노드부터 방문, 연결된 노드를 이어서 방문해서 출력하는 재귀함수를 만들고 싶다면 어떻게 해야할까? 또한, 정점을 방문하고 다시 방문하지 않게 만드려면 어떻게 해야할까?

정답코드

```
#include<bits/stdc++.h>
using namespace std;
const int V = 10;
vector<int> adj[V];
int visited[V];
void go(int idx){
    cout << idx << '\n';
    visited[idx] = 1;
    for(int there : adj[idx]){
        if(visited[there]) continue;
        go(there);
    }
    return;
}
int main(){
    adj[1].push_back(2);
```

```

adj[2].push_back(1);

adj[1].push_back(3);
adj[3].push_back(1);

adj[3].push_back(4);
adj[4].push_back(3);
for(int i = 0; i < V; i++){
    if(adj[i].size() && visited[i] == 0) go(i);
}
}
/*
1
2
3
4
*/

```

인접행렬과 인접리스트의 차이

자아 지금까지 인접행렬과 인접리스트를 배워봤습니다. 둘 중에 어떤 것을 써야 할까요?

일단 차이를 배워보도록 하겠습니다.

공간복잡도

- 인접행렬 : $O(V^2)$
- 인접리스트 : $O(V + E)$

```

// 인접 행렬
bool adj[V][V];
// 인접 리스트
vector<int> adj[V];

```

시간복잡도 : 간선 한개 찾기

- 인접행렬 : $O(1)$
- 인접리스트 : $O(V)$

```

// 인접 행렬

```

```

for(int i = 0; i < V; i++){
    for(int j = 0; j < V; j++){
        if(a[i][j]){
            }
        }
    }
}
// 인접리스트
for(int j = 0; j < adj[i].size(); j++){
    cout << adj[i][j] << " ";
}

```

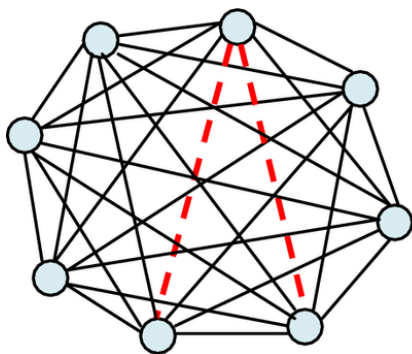
시간복잡도 : 모든 간선찾기

- 인접행렬 : $O(V^2)$
- 인접리스트 : $O(V + E)$

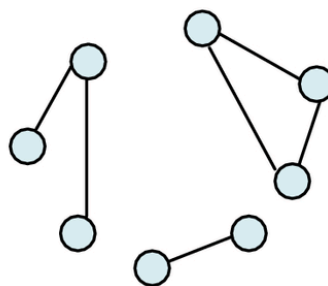
그래프가 희소할 때는 인접리스트, 조밀할 때는 인접행렬을 써라.

그래프가 희소할 때는 인접리스트, 조밀 할 때는 인접행렬이 좋습니다. 그래프가 희소할 때 (sparse)할 때는 인접행렬이 인접리스트보다 메모리를 더 많이 써야 합니다. 간선이 없어서 인접행렬의 대부분의 요소가 0인데도 불구하고 해당 부분을 포함해 2차원 배열을 만들어야 되기 때문입니다.

그래프가 조밀할 때 (dense)할 때는 인접행렬이 인접 리스트보다 더 좋습니다. 어차피 다 연결되어있기 때문에 메모리적 효율성은 동일해지고 정점i에서 정점j까지의 간선이 있는 확인하는 속도가 더 빠르기 때문에 인접행렬이 더 빠릅니다.



(a)



(b)

[a : dense / b : sparse]

정리

보통은 문제에서 sparse한 그래프가 많이 나오기 때문에 인접리스트를 쓰면 됩니다. 다만 문제 또는 코딩인터뷰에서 인접행렬로 주어진다면 그대로 인접행렬로 푸는 것이 좋습니다.

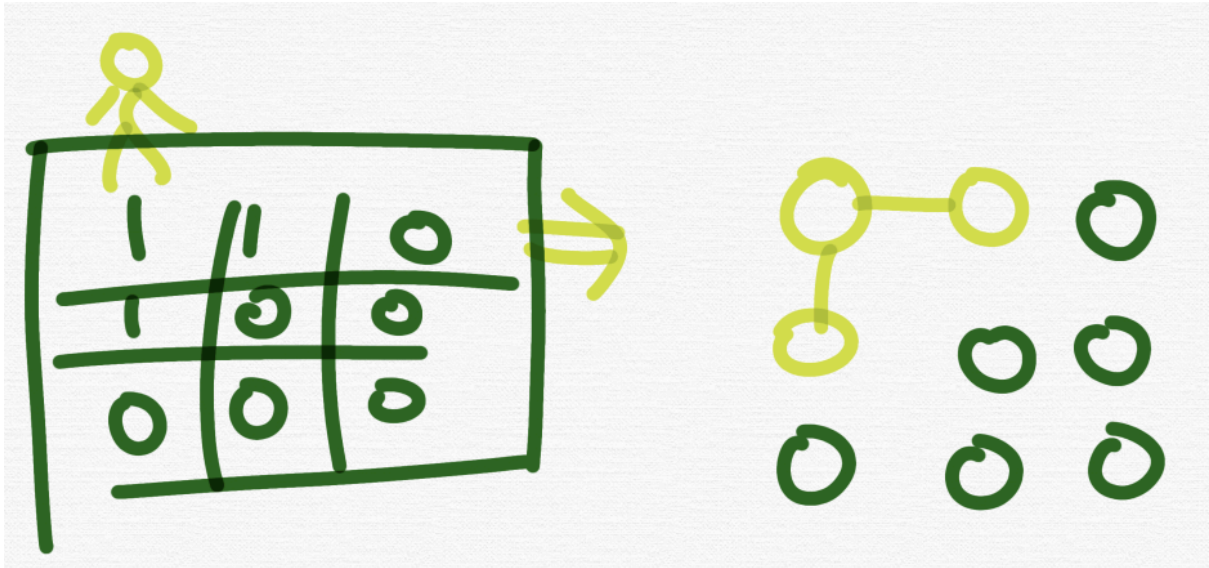
그래프 구현과 탐색 3. 맵과 방향벡터

코딩인터뷰나 알고리즘 문제를 보면 지금까지 배웠던 인접행렬이나 인접리스트로 문제가 주어지는 경우도 있지만 맵으로도 주어지는 경우가 있습니다.

N×M크기의 배열로 표현되는 미로가 있다.

1	0	1	1	1	1
1	0	1	0	1	0
1	0	1	0	1	1
1	1	1	0	1	1

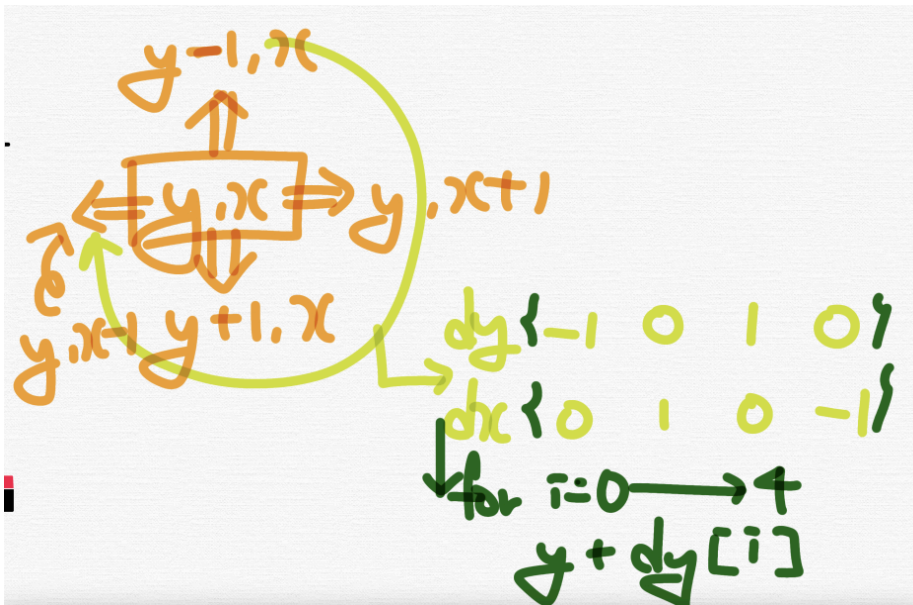
이런 경우에 문제에서 주어진 맵을 받아서 그 맵을 기준으로 탐색을 이어가야 합니다. 또한, 맵은 하나의 그래프입니다. 예를 들어 3 * 3 맵이며 1은 갈 수 있는 지역, 0은 갈 수 없는 지역이며 4방향으로 탐색이 가능하다고 하면 다음과 같은 그래프가 되는 셈입니다.



주의사항 : 이 맵은 인접행렬이 절대 아닙니다!!

4방향탐색과 방향벡터

보통 이 맵을 기준으로 4가지 방향으로 탐색해나아가라고 합니다. 4가지 방향은 보통 위, 아래, 오른쪽, 왼쪽으로 주어집니다.(문제마다 다를 수는 있지만 보통은 이렇게 주어집니다.) 예를 들어 어떠한 y, x 가 주어졌을 때 y, x 를 중심으로 상하좌우 4가지 방향으로 탐색은 어떻게 할까요?



다음과 같이 위쪽부터 시계방향으로 탐색한다고 가정하면 $-1, 0 / 0, 1 / 1, 0 / 0, -1$ 을 더해가며 탐색하게 됩니다.

이를 배열로 나타내면 이런 방향벡터를 정의할 수 있고 이를 기반으로 탐색한다면 이러한 코드가 작성되겠죠? 보통 방향벡터는 dy, dx라는 이름으로 정의합니다. direction_y, direction_x라는 의미입니다.

```
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
for(int i = 0; i < 4; i++){
    ny = y + dy[i];
    nx = x + dx[i];
}
```

자 그러면 문제를 풀면서 이해해보겠습니다.

Q1. {0, 0}좌표에서 dy, dx를 만들어 4방향(위, 오른쪽, 아래, 왼쪽)을 탐색하며 좌표를 출력하시오.

```
#include <bits/stdc++.h>
using namespace std;
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
int main(){
    int y = 0, x = 0;
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        cout << ny << " : " << nx << '\n';
    }
    return 0;
}
/*
-1 : 0
0 : 1
1 : 0
0 : -1
*/
```

Q2. {0, 0}좌표에서 dy, dx를 만들어 8방향(위, 오른쪽, 아래, 왼쪽 및 대각선방향포함)을 탐색하며 좌표를 출력하시오.

만약에 8가지 방향이라면 어떻게 해야할까요? 예를 들어 위쪽, 위쪽오른쪽 .. 등으로 대각선까지 포함해서 문제가 주어진다면 어떻게 해야할까요?

```

#include <bits/stdc++.h>
using namespace std;
const int dy[] = {-1, -1, 0, 1, 1, 1, 0, -1};
const int dx[] = {0, 1, 1, 1, 0, -1, -1, -1};
int main(){
    int y = 0, x = 0;
    for(int i = 0; i < 8; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        cout << ny << " : " << nx << '\n';
    }
    return 0;
}
/*
-1 : 0
-1 : 1
0 : 1
1 : 1
1 : 0
1 : -1
0 : -1
-1 : -1
*/

```

반드시 4방향이 아닐 수 있습니다. 문제에 따라 방향은 달라질 수 있습니다. 이를 주의해서 코드를 구축해야 합니다.

Q3. 3 * 3 맵을 입력받아야 함. 이 맵은 1과 0으로 이루어져있고 {0, 0}은 무조건 1임을 보장한다. {0, 0}부터 4방향을 기준으로 한칸씩 탐색해나가며 방문한 정점은 다시 방문하지 않으며 방문하는 좌표를 출력하는 코드. 0은 갈 수 없는 지역. 1은 갈 수 있는 지역을 구현하시오.

```

#include<bits/stdc++.h>
using namespace std;
const int n = 3;
int a[n][n], visited[n][n];
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};

void go(int y, int x){
    visited[y][x] = 1;
    cout << y << " : " << x << "\n";
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];

```

```

        int nx = x + dx[i];
        if(ny < 0 || ny >= n || nx < 0 || nx >= n) continue;
        if(a[ny][nx] == 0) continue;
        if(visited[ny][nx]) continue;
        go(ny, nx);
    }
    return;
}
int main(){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            cin >> a[i][j];
        }
    }
    go(0, 0);
}
/*
1 0 1
1 0 1
0 1 1
*/

```

우리가 맵을 탐색할 때 다음코드가 무조건 들어가야 합니다. 맵의 주어진 범위를 벗어나지 않게 하는 코드입니다. 이 문제에서 맵의 범위는 0 ~ 3인데 그 밖을 벗어나면 배열 인덱싱에러가 나타나겠죠? 언더플로우와 오버플로우 에러를 이렇게 방지해줍니다. 맵을 탐색할 때는 이렇게 항상 맵을 벗어나는지를 염두해야 합니다.

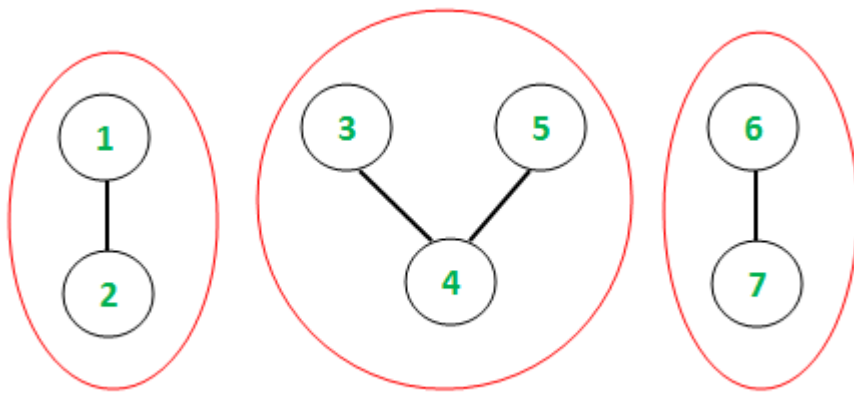
```

if(ny < 0 || ny >= n || nx < 0 || nx >= n) continue;

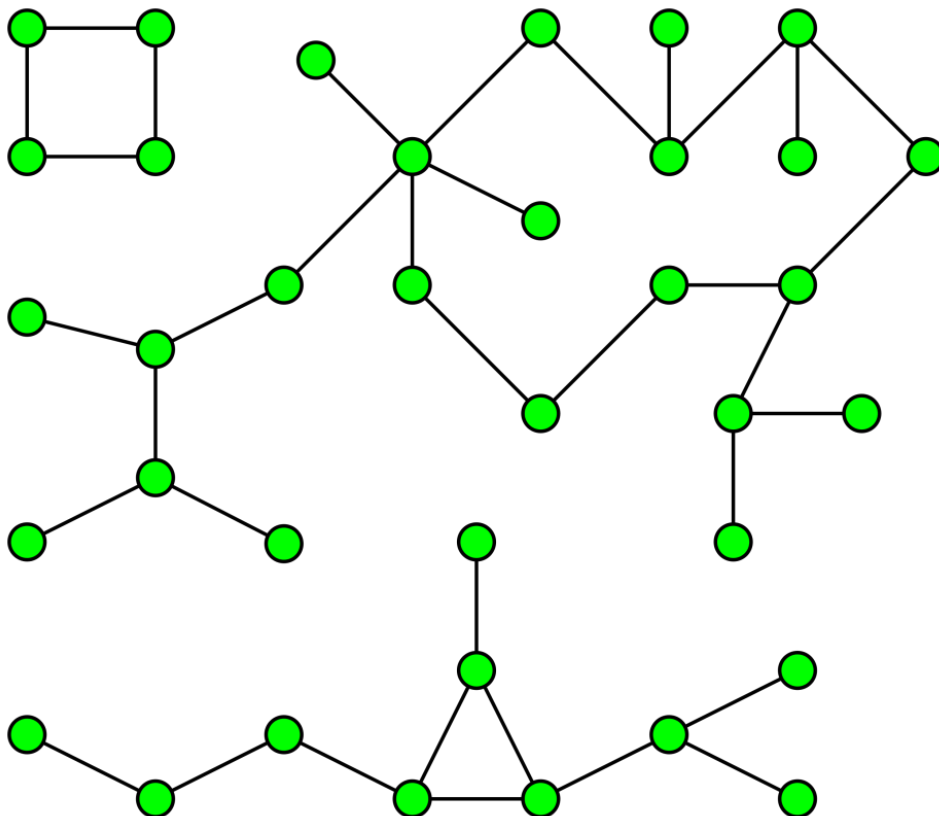
```

연결된 컴포넌트(connected component)

연결된 컴포넌트(connected component)는 연결된 하위그래프를 말하며 연결된 하나의 덩어리라고 보시면 됩니다. 이 덩어리는 연결된 컴포넌트에 속한 "모든 정점을 연결하는 경로가 있다."라는 특징을 가집니다.



앞의 그림에서 연결된 컴포넌트의 수는 총 3개이고 각각의 컴포넌트는 2개, 3개, 2개라는 정점을 가집니다.



앞의 그림에서 connected component는 3개가 됩니다. 즉, 연결되어있는지 아니면 연결되어있지 않는지를 토대로 연결된 컴포넌트로 나눕니다. 이러한 컴포넌트들을 번호를 붙여가며 색칠하는 알고리즘은 폴드필(floodfill)이라고 합니다.

깊이우선탐색

깊이우선탐색, DFS(Depth First Search)는 그래프를 탐색할 때 쓰는 알고리즘이며 어떤 노드부터 시작해 인접한 노드들을 재귀적으로 방문하며 방문한 정점은 다시 방문하지 않으며 각 분기마다 가능한 가장 멀리 있는 노드까지 탐색하는 알고리즘입니다.

DFS의 수도코드

DFS의 수도코드는 다음과 같습니다. 이 수도코드만 이해한다면, DFS 전체를 이해한거나 다름없습니다.

수도코드(pseudocode)는 프로그램의 로직을 표현하기 위해 쓰이는 코드입니다. 주로 어떤 알고리즘이 어떠한 로직을 갖고 있는지 나타내기 위해 씁니다.

```
DFS(u, adj)
  u.visited = true
  for each v ∈ adj[u]
    if v.visited == false
      DFS(v, adj)
```

앞의 코드를 해석하면 다음과 같습니다.

1. 어떠한 정점 u의 visited를 참으로 바꾸고 u로부터 연결되어있는 v지점을 탐색합니다.
2. 이 때 방문되어있지 않은 노드에 대해 재귀적으로 DFS를 호출합니다.

자 다음 그림의 그래프를 DFS로 탐색해본다면 어떻게 될까요?

이를 구현한 코드입니다.

```
#include<bits/stdc++.h>
using namespace std;
const int n = 6;
vector<int> adj[n];
int visited[n];
void DFS(int u){
    visited[u] = 1;
    cout << u << "\n";
    for(int v : adj[u]){
        if(visited[v] == 0){
            DFS(v);
        }
    }
    cout << u << "로부터 시작된 함수가 종료되었습니다.\n";
    return;
}
int main(){
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[4].push_back(2);
    adj[2].push_back(5);
    DFS(1);
}
/*
1
2
4
4로부터 시작된 함수가 종료되었습니다.
5
5로부터 시작된 함수가 종료되었습니다.
2로부터 시작된 함수가 종료되었습니다.
3
3로부터 시작된 함수가 종료되었습니다.
1로부터 시작된 함수가 종료되었습니다.
*/
```

이러한 DFS를 구현하는 방법은 크게 2가지가 있습니다.

구현방법1 : 돌다리를 두들겨 보다.



```
void DFS(int here){  
    visited[here] = 1;  
    for(int there : adj[here]){  
        if(visited[there]) continue;  
        DFS(there);  
    }  
}
```

만약 방문이 되어있다면(visited[there])이라면 방문을 하지 않고 방문이 안되어있다면 방문을 하는 코드입니다.

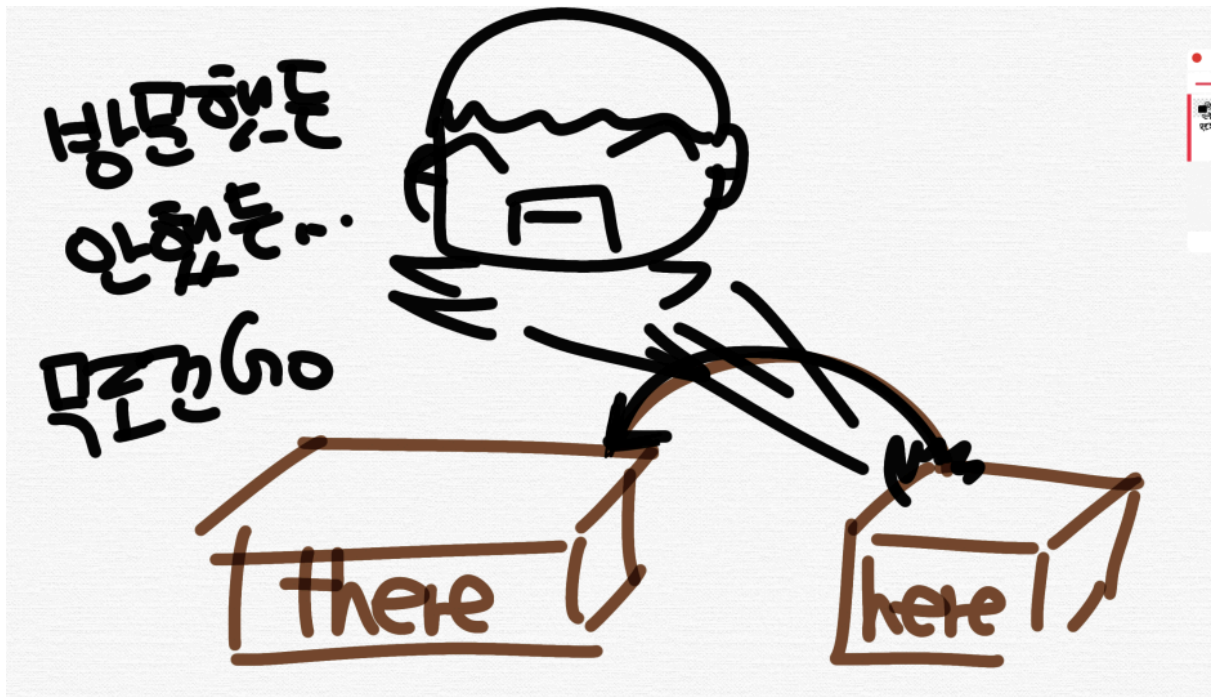
앞의 코드는 다음과 같이 구현해도 됩니다.

```
void DFS(int here){  
    for(int there : adj[here]){  
        if(visited[there]) continue;  
        visited[there] = 1;  
        DFS(there);  
    }  
}
```

다만 이럴 경우 시작지점에 대해 방문처리를 해주어야 합니다. 예를 들어 1, 2, 3, 4 지점을 탐색한다고 하고 1지점부터 탐색을 이어나간다고 했을 때 다음과 같이 코드를 구축해야 합니다.

```
visited[1] = 1;  
DFS(1);
```

구현방법2 : 못 먹어도 GO



일단 방문되어있던 안되어있던 무조건 DFS를 호출하고 해당 함수에서 만약에 방문되어있다면 return 해 함수를 종료시키는 방법입니다.

```
void DFS(int here){  
    if(visited[here]) return;  
    visited[here] = 1;  
    for(int there : adj[here]){  
        DFS(there);  
    }  
}
```

문제에 따라 1번방법이 편할 때도 2번방법이 편할 때도 있습니다. 두 개의 코드 모두 자유롭게 구현할 줄 알아야 합니다.

Q. 종화는 방구쟁이야!



종화는 21세기 유명한 방구쟁이다. 종화는 방구를 길 때 "이러다... 다 죽어!!" 를 외치며 방구를 끼는데 이렇게 방귀를 끼었을 때 방귀는 상하좌우 네방향으로 뿔어나가며 종화와 연결된 "육지"는 모두 다 오염된다. "바다"로는 방구가 갈 수 없다. 맵이 주어졌을 때 종화가 "이러다... 다 죽어!!"를 "최소한" 몇 번외쳐야 모든 육지를 오염시킬 수 있는지 말해보자. 1은 육지이며 0은 바다를 가리킨다.

입력

맵의 세로길이 N과 가로길이 M 이 주어지고 이어서 $N * M$ 의 맵이 주어진다.

출력

"이러다... 다 죽어!!"를 몇 번 외쳐야하는지 출력하라.

범위

$1 \leq N \leq 100$

$1 \leq M \leq 100$

예제입력

5 5

1 0 1 0 1

1 1 0 0 1

0 0 0 1 1

0 0 0 1 1

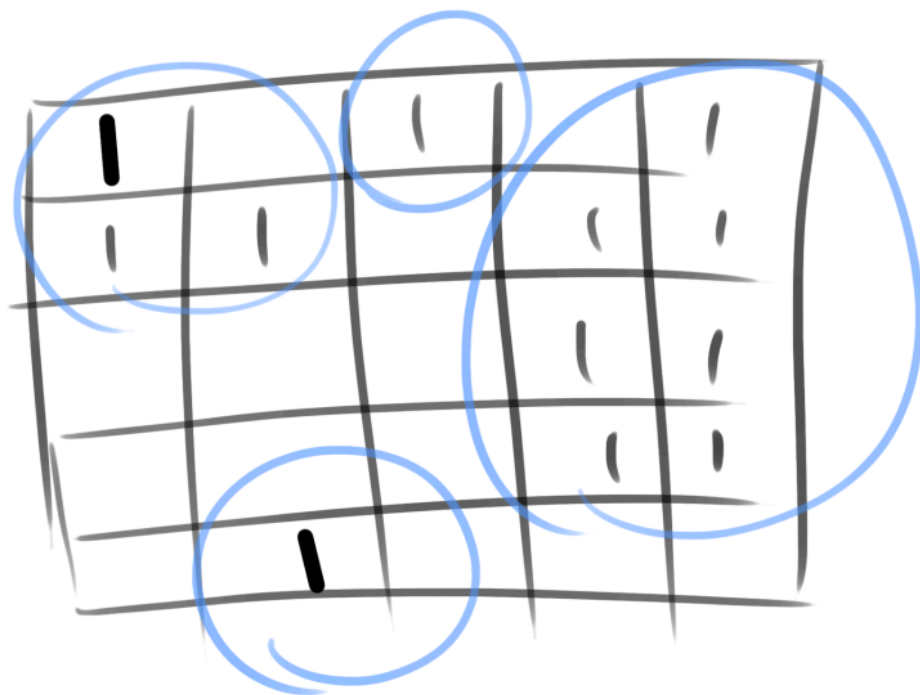
0 1 0 0 0

예제출력

4

문제해설

이 문제는 어떤 문제일까요? 바로 앞서 설명한 연결된 컴포넌트(connected component)를 찾는 문제입니다.



예제입력을 시각화했을 때 저렇게 묶은 부분이 바로 연결된 컴포넌트이며 이러한 덩어리가 몇개인지를 찾는 문제입니다. 그렇다면 어떻게 찾을 수 있을까요? 간단하게 DFS를 구축하면 됩니다.

정답코드

```
#include<bits/stdc++.h>
using namespace std;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int m, n, k, y, x, ret, ny, nx, t;
int a[104][104];
bool visited[104][104];
void DFS(int y, int x){
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >= n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
            DFS(ny, nx);
        }
    }
    return;
}

int main(){
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n >> m;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
        }
    }
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(a[i][j] == 1 && !visited[i][j]){
                ret++; DFS(i, j);
            }
        }
    }
    cout << ret << '\n';
    return 0;
}
```

상세설명

```
if(a[i][j] == 1 && !visited[i][j]){
    ret++; DFS(i, j);
}
```

방문을 하지 않았고 육지라면 ret++를 하고 DFS를 호출합니다.

```
void DFS(int y, int x){
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >= n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
            DFS(ny, nx);
        }
    }
    return;
}
```

메인이 되는 DFS입니다. 방문했다고 표기하고 4방향을 탐색하면서 DFS를 탐색해나갑니다.

디버깅 팁

이 문제를 풀 때 DFS가 퍼져나가는 것이 상상이 안된다면 이렇게 해보시는 것은 어떨까요?

```
#include<bits/stdc++.h>
using namespace std;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int m, n, k, y, x, ret, ny, nx, t;
int a[104][104];
bool visited[104][104];
void DFS(int y, int x){
    cout << y << " : " << x << '\n';
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >= n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
```

```

        DFS(ny, nx);
    }
}
return;
}

int main(){
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n >> m;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
        }
    }
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(a[i][j] == 1 && !visited[i][j]){
                ret++; DFS(i, j);
            }
        }
    }
    cout << ret << '\n';
    return 0;
}

```

이 코드는 DFS 함수 내에 아래와 같은 코드를 집어넣었습니다.

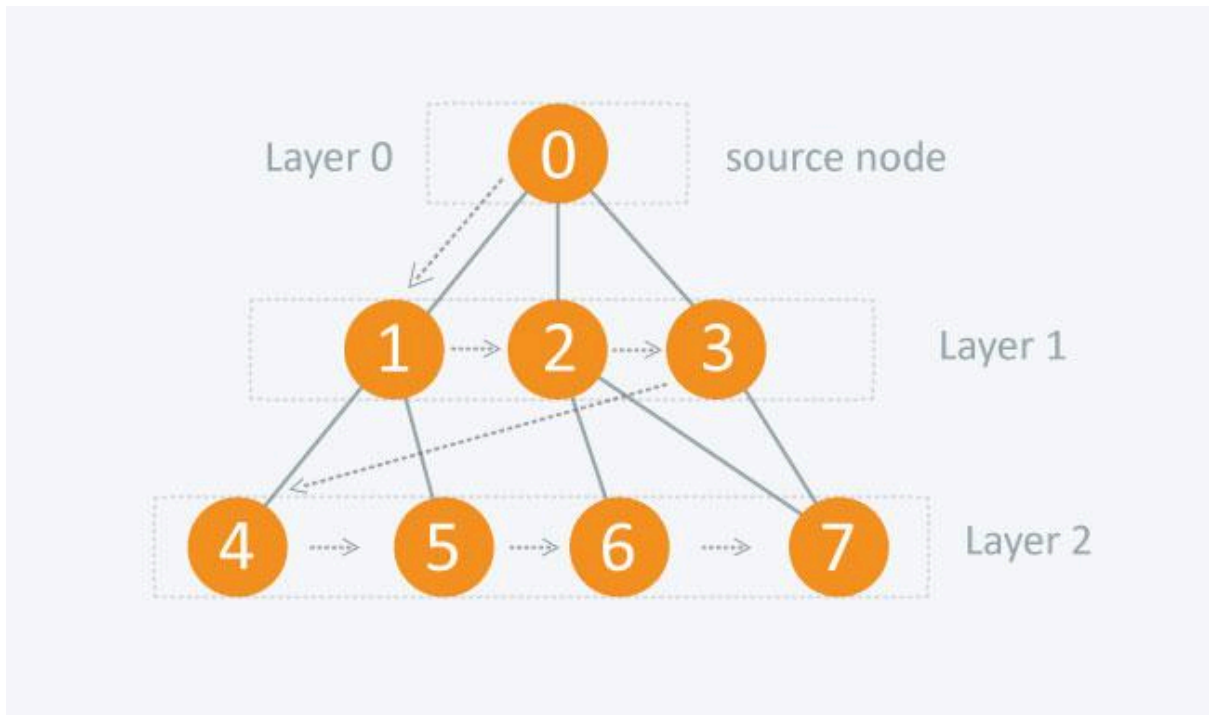
```
cout << y << " : " << x << '\n';
```

이렇게 y, x가 어떻게 뺄어나가는지를 보면서 이해를 하시는 것이 좋습니다.

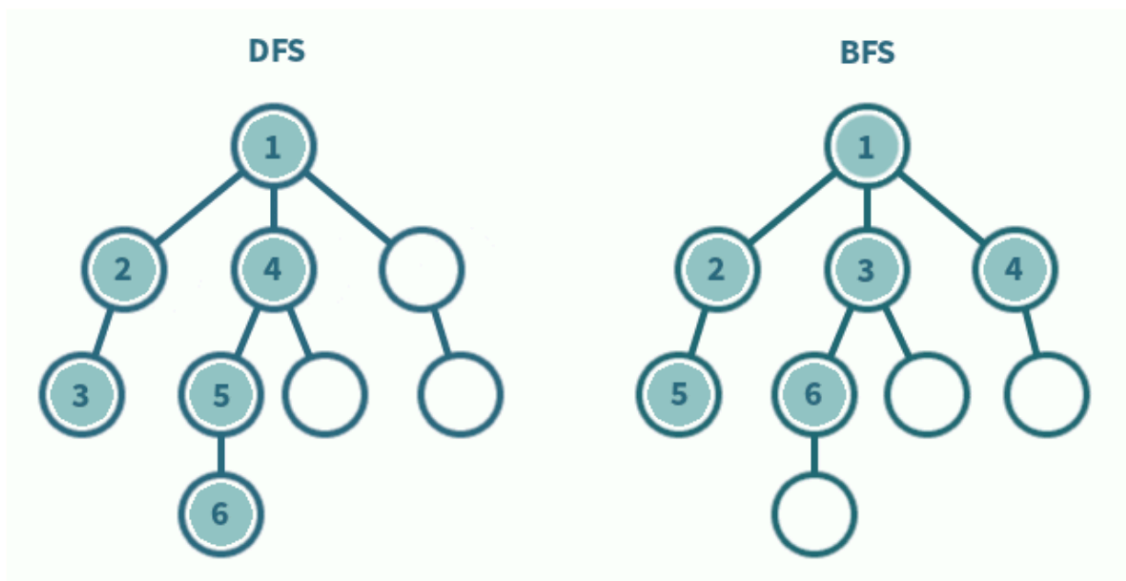
너비우선탐색(BFS, Breadth First Search)

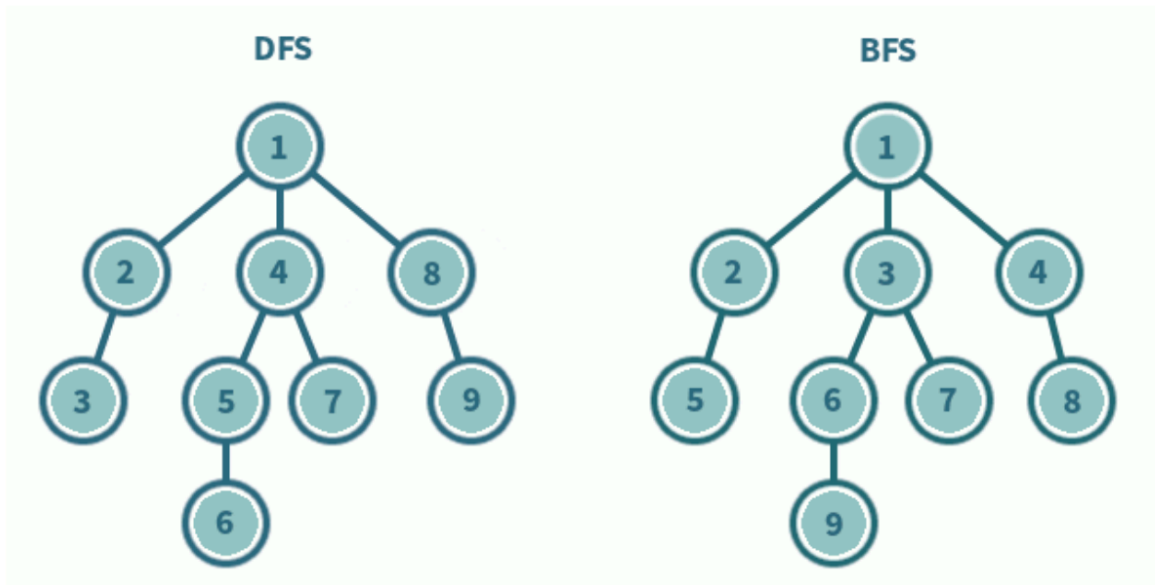
BFS는 그래프를 탐색하는 알고리즘이며 어떤 정점에서 시작해 다음 깊이의 정점으로 이동하기전 현재 깊이의 모든 정점을 탐색하며 방문한 정점은 다시 방문하지 않는 알고리즘입니다. 같은 가중치를 가진 그래프에서 최단거리알고리즘으로 쓰입니다.

BFS로 탐색을 한다는 것은 다음그림처럼 레이어별, 레벨별로 탐색한다는 뜻입니다.



DFS와 BFS의 탐색차이는 다음과 같습니다.





BFS의 수도코드1

먼저 시작지점인 u 를 "방문처리"를 하고 "큐(queue)에 푸시"를 합니다. 그리고 $q.size()$ 만큼 while 반복문을 돌면서 큐 앞단의 있는 u 를 다시 끄집어내서 그 u 를 중심으로 인접한 노드들을 탐색을 합니다. 방문한 정점은 다시 방문하지 않고 방문처리를 하면서 큐(queue)에 푸시를 하며 방문처리를 합니다.

```

BFS(G, u)
    u.visited = true
    q.push(u);
    while(q.size())
        u = q.front()
        q.pop()
        for each v ∈ G.Adj[u]
            if v.visited == false
                v.visited = true
                q.push(v)

```

BFS의 수도코드2

수도코드1은 방문처리만을 하는 코드입니다. 하지만 보통 문제에서 BFS를 쓸 때는 가중치가 같은 그래프 내에서 최단거리 알고리즘으로 쓰입니다. 그러기 위해 "최단거리 배열"을 방문하면서 만들어주어야 하는데 다음과 같은 수도코드를 사용합니다.

```

BFS(G, u)
    u.visited = 1

```

```

q.push(u);
while(q.size())
    u = q.front()
    q.pop()
    for each v ∈ G.Adj[u]
        if v.visited == false
            v.visited = u.visited + 1
            q.push(v)

```

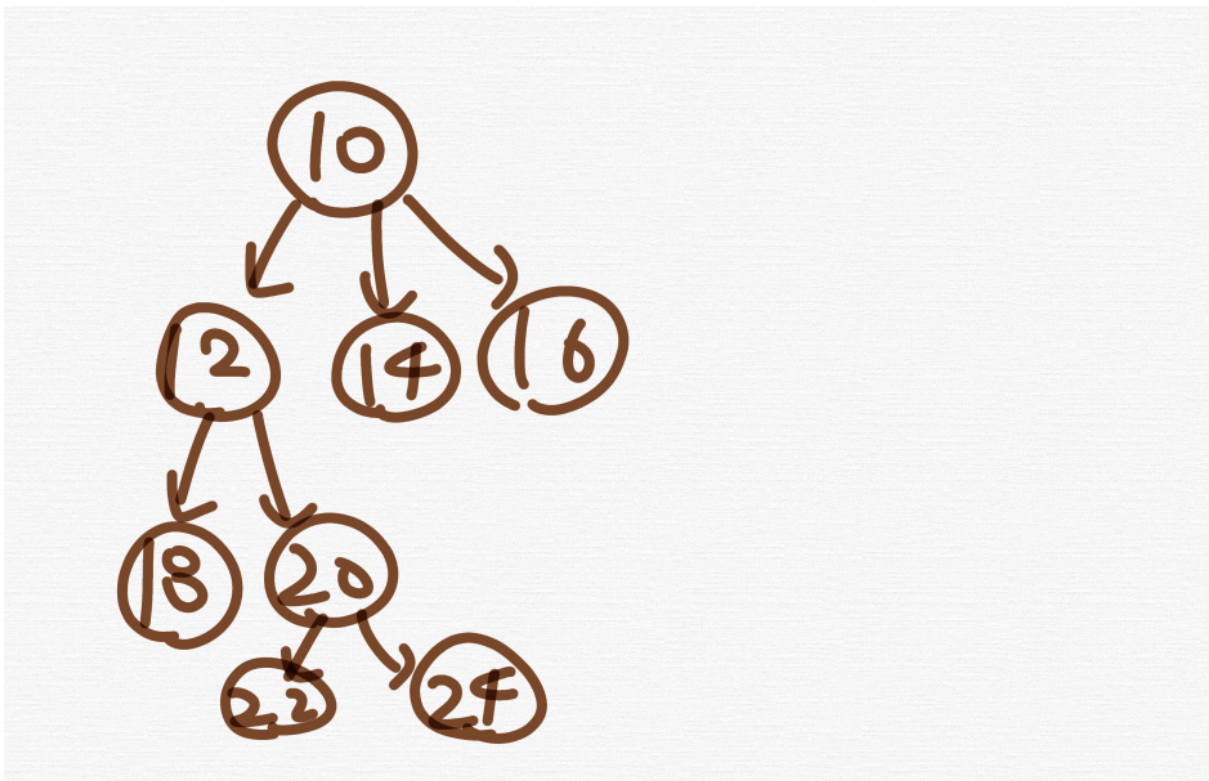
차이점은 이 코드 한 줄입니다.

```

v.visited = u.visited + 1

```

자, 이 그래프에서 화살표는 한칸이라는 의미를 가집니다. 10번정점으로부터 24번 정점까지의 최단거리는 얼마일까요?



답은 3칸입니다. 이를 BFS로 구현해보면서 구해보겠습니다.

앞의 그래프를 인접리스트로 구현하고 10번정점 부터 탐색하는 BFS 코드입니다.

그림과 대조하면서 확인해주세요.

```

#include<bits/stdc++.h>
using namespace std;
vector<int> adj[100];
int visited[100];
int nodeList[] = {10, 12, 14, 16, 18, 20, 22, 24};
void BFS(int here){
    queue<int> q;
    visited[here] = 1;
    q.push(here);
    while(q.size()){
        int here = q.front(); q.pop();
        for(int there : adj[here]){
            if(visited[there]) continue;
            visited[there] = visited[here] + 1;
            q.push(there);
        }
    }
}
int main(){
    adj[10].push_back(12);
    adj[10].push_back(14);
    adj[10].push_back(16);

    adj[12].push_back(18);
    adj[12].push_back(20);

    adj[20].push_back(22);
    adj[20].push_back(24);
    BFS(10);
    for(int i : nodeList){
        cout << i << " : " << visited[i] << '\n';
    }
    cout << "10번으로부터 24번까지 최단거리는 : " << visited[24] - 1 <<
    '\n';
    return 0;
}
/*
10 : 1
12 : 2
14 : 2
16 : 2
18 : 3
20 : 3
22 : 4
*/

```

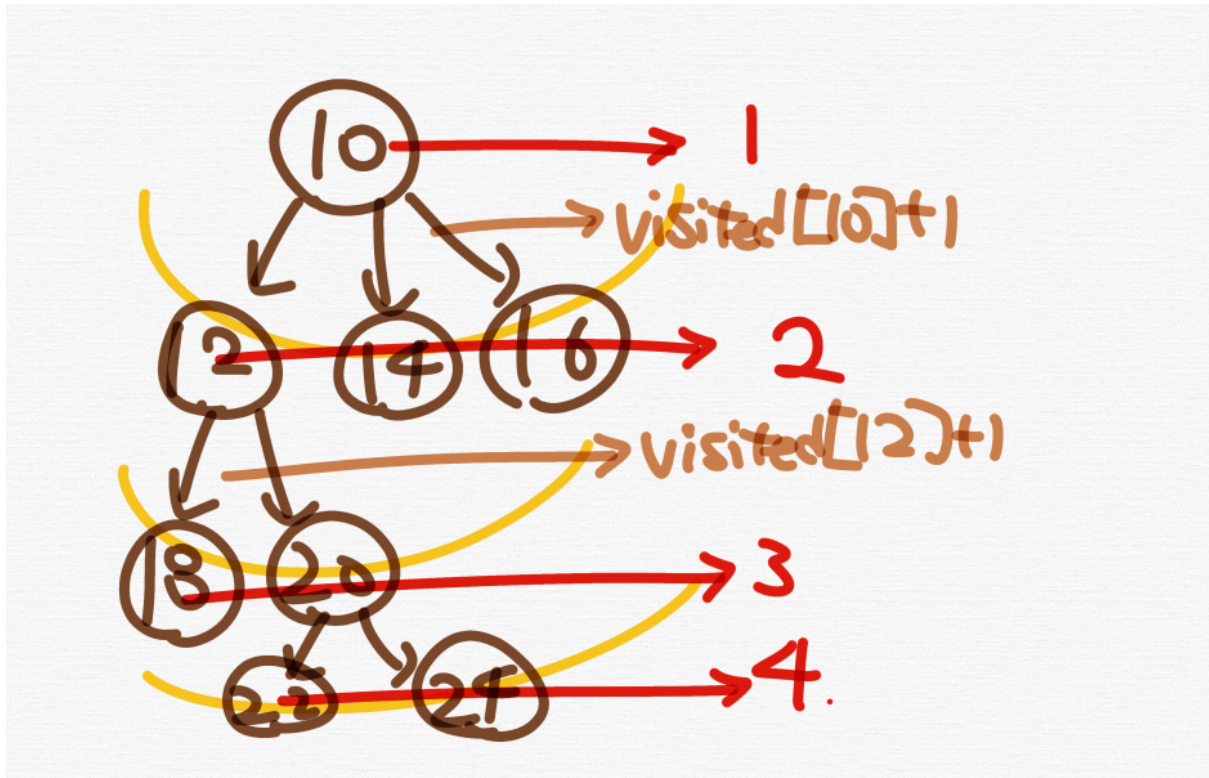
24 : 4

10번으로부터 24번까지 최단거리는 : 3

*/

탐색원리는 다음과 같습니다.

이런식으로 탐색하게 되며 자동적으로 최단거리가 구해지는 것이죠.

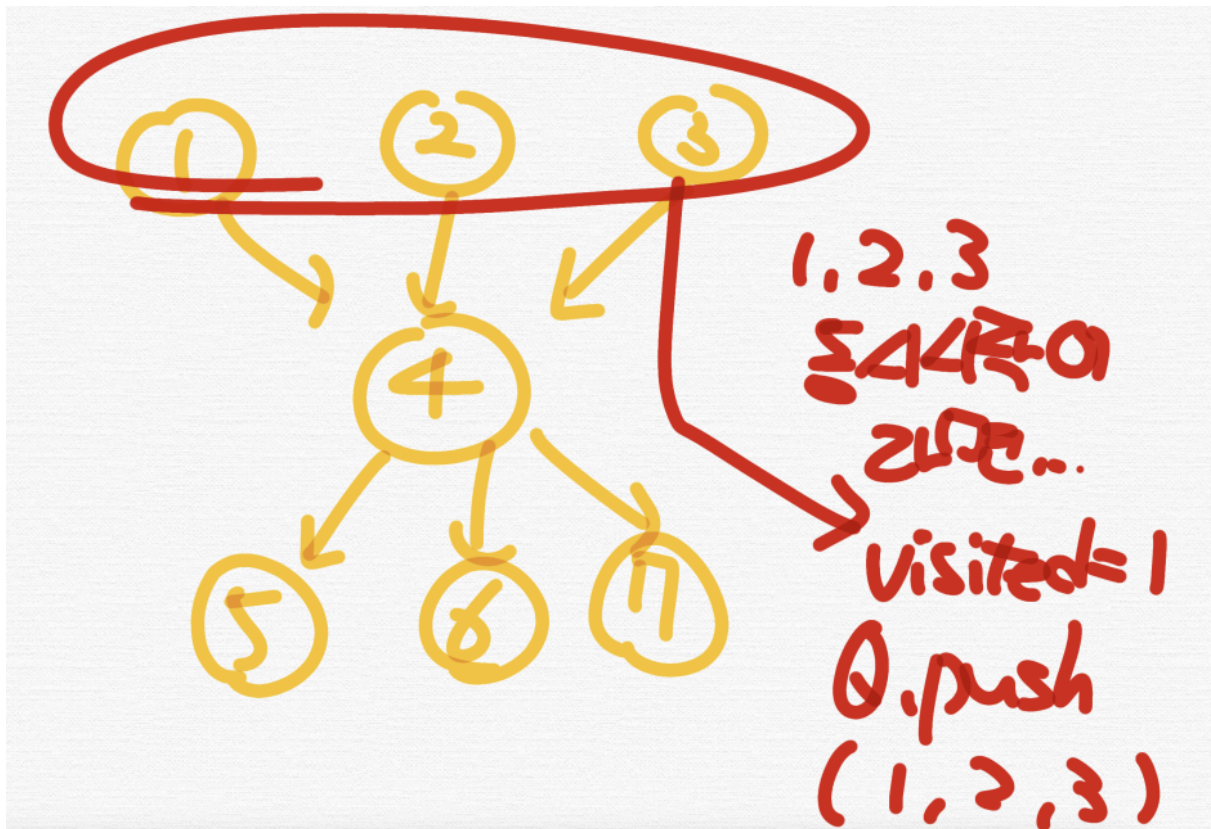


Q. 만약 앞의 그래프의 가중치가 한칸이 아니고 2칸이라면 어떻게 될까요?

3 * 2를 하면 됩니다. 답은 6칸이 되겠죠?

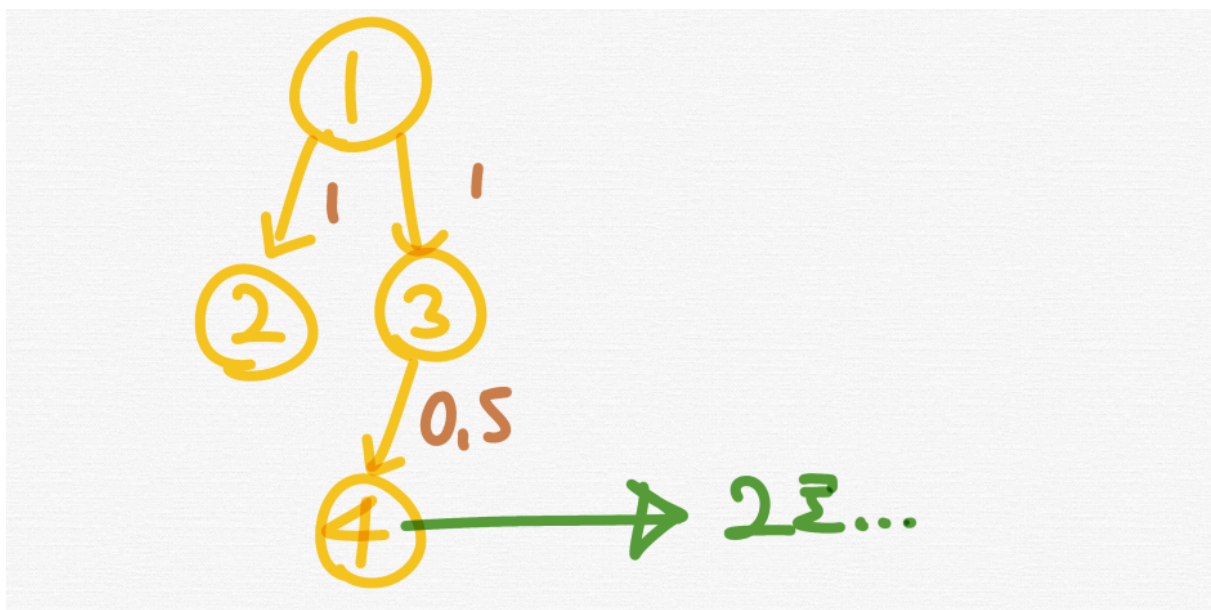
Q. 시작지점이 다수라면?

"시작지점이 다수" 라면 처음 큐에 푸시하는 지점도 다수가 되어야 하며 해당 지점들의 visited를 모두 1로 만들면서 시작해야 합니다.



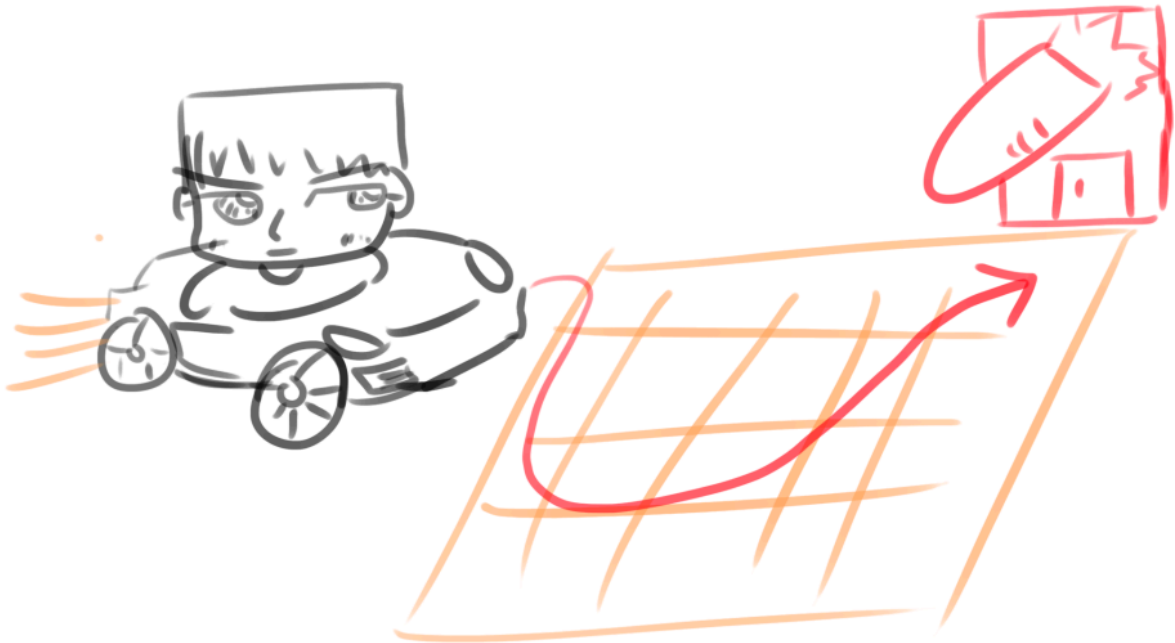
Q. BFS는 왜 가중치가 같은 그래프내에서만 최단거리 알고리즘으로 쓰이나요? 가중치가 다른 그래프에서는 안되나요?

다음과 같은 그래프가 있다고 칩시다. 3번에서 4번으로 가는 가중치가 0.5이며 그래프내의 가중치가 서로 다르게 되죠? 이 때 1번부터 4번까지 가는 최단거리를 구한다고 했을 때 우리가 지금까지 배운 BFS로는 2밖에 출력이 안되게 됩니다. 그래서 못쓰는 것이죠.



[참고] 가중치가 다른 그래프 내에서 최단거리 알고리즘은 다익스트라, 벨만포드 등을 써야 합니다.

Q. 당근마켓 엔지니어 승원이



승원이는 당근을 좋아해서 당근마켓에 엔지니어로 취업했다. 당근을 매우좋아하기 때문에 차도 당근차를 샀다. 이 당근차는 한칸 움직일 때마다 당근을 내뿜으면서 간다. 즉, "한칸" 움직일 때 "당근한개"가 소모된다는 것이다. 승원이는 오늘도 아침 9시에 일어나 당근마켓으로 출근하고자 한다. 승원이는 최단거리로 당근마켓으로 향한다고 할 때 당근몇개를 소모해야 당근마켓에 갈 수 있는지 알아보자. 이 때 승원이는 육지로만 갈 수 있으며 바다로는 못간다. 맵의 1은 육지이며 0은 바다를 가리킨다. 승원이는 상하좌우로만 갈 수 있다.

입력

맵의 세로길이 N 과 가로길이 M 이 주어지고 이어서 $N * M$ 의 맵이 주어진다. 그 다음 줄에 승원이의 위치(y, x)와 당근마켓의 위치(y, x)가 주어진다. 이 때 승원이의 시작위치(y, x)에서 "당근한개"가 이미 소모된 상태로 본다.

출력

당근을 몇개 소모해야 하는지 출력하라.

범위

1 <= N <= 100

1 <= M <= 100

예제입력

5 5

0 0

4 4

1 0 1 0 1

1 1 1 0 1

0 0 1 1 1

0 0 1 1 1

0 0 1 1 1

예제출력

9

해설

이 문제는 어떤 문제일까요? 바로 "가중치가 같은 그래프 내의 최단거리알고리즘" 문제입니다. 한칸씩 이동할 때마다 당근 한개를 쓰기 때문에 가중치가 같은 그래프이며 여기서 최단거리를 구하는 알고리즘이기 때문이죠. 그렇다면 BFS를 통해 최단거리 배열을 만들어서 해야 합니다.

정답코드

```
#include<bits/stdc++.h>
using namespace std;
const int max_n = 104;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int n, m, a[max_n][max_n], visited[max_n][max_n], y, x, sy, sx, ey, ex;
int main(){
    scanf("%d %d", &n, &m);
    cin >> sy >> sx;
```

```

cin >> ey >> ex;
for(int i = 0; i < n; i++){
    for(int j = 0; j < m; j++){
        cin >> a[i][j];
    }
}
queue<pair<int, int>> q;
visited[sy][sx] = 1;
q.push({sy, sx});
while(q.size()){
    tie(y, x) = q.front(); q.pop();
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        if(ny < 0 || ny >= n || nx < 0 || nx >= m || a[ny][nx] == 0)
continue;

        if(visited[ny][nx]) continue;
        visited[ny][nx] = visited[y][x] + 1;
        q.push({ny, nx});
    }
}
printf("%d\n", visited[ey][ex]);
// 최단거리 디버깅
for(int i = 0; i < n; i++){
    for(int j = 0; j < m; j++){
        cout << visited[i][j] << ' ';
    }
    cout << '\n';
}
return 0;
}

```

상세설명

```

queue<pair<int, int>> q;
visited[sy][sx] = 1;
q.push({sy, sx});
while(q.size()){
    tie(y, x) = q.front(); q.pop();
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        if(ny < 0 || ny >= n || nx < 0 || nx >= m || a[ny][nx] == 0)
continue;

        if(visited[ny][nx]) continue;
        visited[ny][nx] = visited[y][x] + 1;
    }
}

```

```

        q.push({ny, nx});
    }
}

```

2차원배열이기 때문에 y, x를 담기 위해 pair를 정의하고 visited 배열의 값을 1로 설정한 뒤 시작지점을 큐에 푸시를 합니다. 그 이후 큐의 크기만큼 while 반복문을 돌며 탐색을 진행합니다. 큐의 가장 앞부분에 있는 좌표를 끄집어내어 탐색을 다시 이어가며 4방향을 탐색하며 다시 visited배열을 갱신하며 큐에 넣으며 진행합니다.

여기서 핵심은 최단거리배열을 만드는 것입니다.

```
visited[ny][nx] = visited[y][x] + 1;
```

이렇게 내가 가야할 곳인 ny, nx에 지금 있는 곳 y, x로부터 + 1를 하면서 최단거리배열을 쌓다보면 어떻게 될까요? 바로 이렇게 쌓이게 됩니다.

```

9
1 0 5 0 9
2 3 4 0 8
0 0 5 6 7
0 0 6 7 8
0 0 7 8 9

```

이렇게 최단거리배열을 만들었을 뿐인데 -> "해당 목표좌표를 출력"하니 그 부분에 해당하는 최단거리값이 출력되는 것을 볼 수 있죠? 이렇게 BFS는 최단거리배열만 잘 만들면 됩니다.

DFS와 BFS비교

둘 다 시간복잡도는 인접리스트로 이루어졌다면 $O(V + E)$ 이고 인접행렬의 경우 $O(V^2)$ 가 되는 것은 동일하며 다음과 같은 차이가 있습니다.

이름	설명
DFS	메모리를 덜 씀. 절단점 등 구할 수 있음. 코드가 좀 더 짧으며 완전탐색의 경우에 많이 씀.
BFS	메모리를 더 씀. 가중치가 같은 그래프내에서 최단거리를 구할 수 있음. 코드가 더 김

문제에서 "퍼져나간다", "탐색한다" 이 2글자가 있으면 반드시 DFS, BFS가 생각이 나야 합니다. 한쪽은 깊이우선탐색이고 한쪽은 너비우선탐색이라 서로의 길이 다르지만 뭐 어떻습니까. 모두다 퍼져나가거나 탐색할 때 쓰이는 알고리즘인 걸요.

트리 순회

트리 순회(Tree traversal)는 트리 구조에서 각각의 노드를 정확히 한 번만, 체계적인 방법으로 방문하는 과정을 말합니다. 이는 노드를 방문하는 순서에 따라 후위순회, 전위순회, 중위순회, 레벨순회가 있습니다. 보통 설명할 때는 이진트리를 기반으로 설명하지만 다른 모든 트리에서 일반화를 시킬 수 있습니다.

후위순회

후위순회(postorder traversal)는 자식들 노드를 방문하고 자신의 노드를 방문하는 것을 말합니다.

[참고] post : ~이후의를 뜻하는 접두사

수도코드

```
postorder( node )
    if (node.visited == false)
        postorder( node->left )
        postorder( node->right )
        node.visited = true
```

방문한다라는 것은 visited 배열을 색칠하는 것을 생각해주세요.

전위순회

전위순회(preorder traversal)는 먼저 자신의 노드를 방문하고 그 다음 노드들을 방문하는 것을 말합니다. (DFS를 생각하면 됩니다.)

[참고] pre : ~이전의를 뜻하는 접두사

수도코드

```
preorder( node )
    if (node.visited == false)
        node.visited = true
        preorder( node->left )
        preorder( node->right )
```

중위순회

중위순회(inorder traversal)는 왼쪽 노드를 먼저 방문 그다음의 자신의 노드를 방문하고 그 다음 오른쪽 노드를 방문하는 것을 말합니다. 안에 끼워넣다. 를 생각해야 합니다.

수도코드

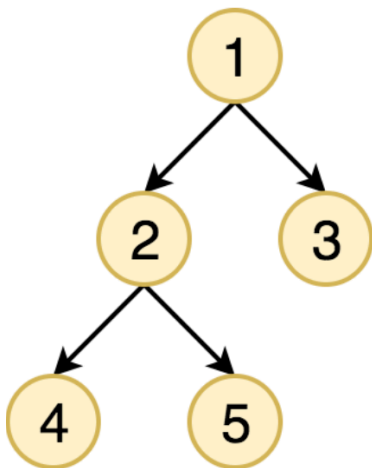
```
inorder( node )
    if (node.visited == false)
        inorder( node->left )
        node.visited = true
        inorder( node->right )
```

레벨순회

레벨순회(level traversal)입니다. 앞서 설명한 BFS를 생각하시면 됩니다.

Q. 아래의 그래프가 주어졌을 때 preorder, inorder, postorder를 구현하라.

수도코드를 보고 코드를 구축하는 연습을 하셔야 합니다. 아래와 같이 그래프가 주어졌을 때 비어진 함수를 채워서 preorder, inorder, postorder를 구현하고 이 때 방문했을 때 해당 노드를 출력하는 것을 구현해보세요.



```
#include <bits/stdc++.h>
```

```

using namespace std;
vector<int> adj[1004];
int visited[1004];

void postOrder(int here){
}
void preOrder(int here){
}
void inOrder(int here){
}

int main(){
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[2].push_back(5);
    int root = 1;
    cout << "\n 트리순회 : postOrder \n";
    postOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : preOrder \n";
    preOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : inOrder \n";
    inOrder(root);
    return 0;
}

```

정답코드

```

#include <bits/stdc++.h>
using namespace std;
vector<int> adj[1004];
int visited[1004];

void postOrder(int here){
    if(visited[here] == 0){
        if(adj[here].size() == 1)postOrder(adj[here][0]);
        if(adj[here].size() == 2){
            postOrder(adj[here][0]);
            postOrder(adj[here][1]);
        }
        visited[here] = 1;
        cout << here << ' ';
    }
}

void preOrder(int here){

```

```

        if(visited[here] == 0){
            visited[here] = 1;
            cout << here << ' ';
            if(adj[here].size() == 1)preOrder(adj[here][0]);
            if(adj[here].size() == 2){
                preOrder(adj[here][0]);
                preOrder(adj[here][1]);
            }
        }
    }
}

void inOrder(int here){
    if(visited[here] == 0){
        if(adj[here].size() == 1){
            inOrder(adj[here][0]);
            visited[here] = 1;
            cout << here << ' ';
        }else if(adj[here].size() == 2){
            inOrder(adj[here][0]);

            visited[here] = 1;
            cout << here << ' ';

            inOrder(adj[here][1]);
        }else{
            visited[here] = 1;
            cout << here << ' ';
        }
    }
}

}

int main(){
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[2].push_back(5);
    int root = 1;
    cout << "\n 트리순회 : postOrder \n";
    postOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : preOrder \n";
    preOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : inOrder \n";
    inOrder(root);
    return 0;
}
/*

```

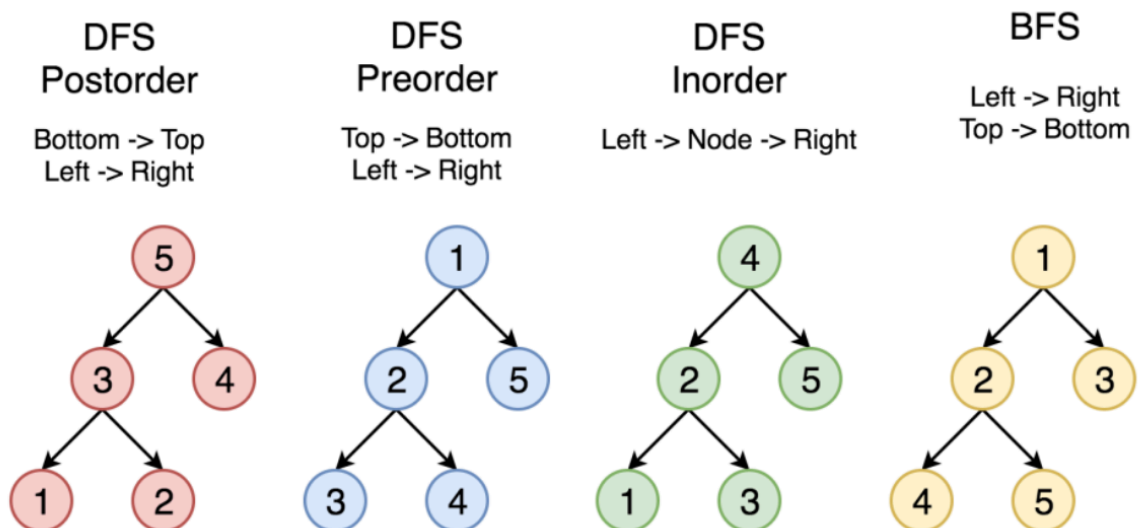
```

트리순회 : postOrder
4 5 2 3 1
트리순회 : preOrder
1 2 4 5 3
트리순회 : inOrder
4 2 5 1 3
*/

```

정리

수도코드처럼 어떤 부분을 "방문처리를 할까?"만을 생각하면 됩니다. 사실 이 그래프는 단방향간선이기 때문에 visited배열은 쓰지 않아도 됩니다. 하지만 수도코드에 있기 때문에 넣어서 구현해봤습니다.



앞의 그림처럼 postorder의 경우 4, 5, 2, 3, 1 이런식으로 방문되는 것을 볼 수있죠? 앞의 그림과 비교하면서 구현하는 것을 연습해 나가시면 됩니다.